VOLUME I OF IV · FOUNDATIONS

SQL
Zero to Advanced

A complete textbook, practical course, interview handbook and coding workbook — written for readers with no prior knowledge.

Primary dialect: MySQL 8+

Dialect notes for PostgreSQL, SQL Server and Oracle throughout

For students, freshers, developers, analysts, engineers and placement preparation

PARTS ONE TO TWELVE · DATABASES, SETUP, COMMANDS, TYPES, TABLES, KEYS, DML, SELECT, FILTERING, SORTING, OPERATORS, FUNCTIONS

How to read this book

This book is written to be read front to back by someone who has never opened a database in their life, and to be re-opened later as a reference by someone preparing for an interview the following morning. Every chapter follows the same rhythm, so you always know where you are:

1. **Concept** — what the thing is, in plain words, before any code.
2. **Explanation** — why it exists and what problem it solves.
3. **Syntax** — the exact shape of the statement, with every optional clause marked.
4. **Example** — runnable SQL against a realistic dataset.
5. **Output** — the actual result set you should see.
6. **Real-world use** — where a working engineer meets this.
7. **Common mistake** — the error that costs people marks and production incidents.
8. **Interview question** — asked in real screens, with the answer.
9. **Practice** — work to do before moving on.

Difficulty is labelled on every practice item and every interview question: Beginner Intermediate Advanced. Nothing later in the book depends on you having solved the advanced items early.

The dataset used throughout

Rather than inventing new tables for every example, this book uses one small company database, extended when a chapter needs more. Build it once and every example in Volumes I and II will run. The full script appears in Part Two; the shape is:

TABLE	MEANING	KEY COLUMNS
departments	Business units	dept_id (PK), dept_name, location
employees	Staff records	emp_id (PK), name, dept_id (FK), manager_id (FK, self), salary, hire_date
customers	People who buy	cust_id (PK), name, city, signup_date
products	Things sold	product_id (PK), product_name, category, price
orders	Purchases	order_id (PK), cust_id (FK), order_date, status, amount
order_items	Lines on an order	order_id (FK), product_id (FK), qty, unit_price

IMPORTANT

Type the queries. Do not read them. SQL is a language of small, precise mistakes — a missing `GROUP BY` column, a `NULL` where you expected zero — and you only learn to see them by making them yourself.

Contents of the four volumes

Volume I — Foundations (this volume)

Part 1	SQL and database basics
Part 2	Setting up MySQL, Workbench and the command line
Part 3	SQL command families: DDL, DML, DQL, DCL, TCL
Part 4	Data types
Part 5	Creating and managing tables
Part 6	Keys and constraints
Part 7	INSERT, UPDATE and DELETE
Part 8	SELECT from zero
Part 9	Filtering data and the truth about NULL
Part 10	Sorting, limiting and pagination
Part 11	Operators and precedence
Part 12	The complete function reference

Volume II — Querying and design

Part 13	GROUP BY and HAVING
Part 14	Joins — the master chapter
Part 15	Set operators
Part 16	Subqueries
Part 17	Common table expressions and recursion
Part 18	Window functions
Part 19	CASE expressions and conditional aggregation
Part 20	Database design and ER modelling
Part 21	Normalization, 1NF to 5NF
Part 22	Views
Part 23	Indexing
Part 24	Query optimization

Volume III — Engineering and projects

Part 25	Transactions and ACID
Part 26	Concurrency, isolation, locks and deadlocks
Part 27	Stored procedures
Part 28	User-defined functions
Part 29	Triggers
Part 30	Temporary tables and derived tables
Part 31	JSON in SQL
Part 32	Security, users, roles and privileges
Part 33	SQL injection and defence
Part 34	Partitioning, sharding, replication, CAP
Part 35	SQL from Python, Java, Spring Boot, Node.js and PHP
Part 36	Ten complete real-world projects
Part 37	Query patterns and their solving approach

Volume IV — Practice, interview and revision

Part 38	300+ graded practice problems with solutions
Part 39	450 interview questions with answers
Part 40	Comparison tables for fast revision
Part 41	Tricks, traps and common mistakes
Part 42	Cheat sheets
Part 43	7, 15, 30, 60 and 90-day roadmaps

PART ONE

SQL and database basics

Before a single query, the vocabulary. Every word in this part is used for the rest of the book, so nothing here is optional.

1.1 What is data?

Data is a recorded fact. The number *45000* is data. On its own it means nothing: it could be a salary, a population, a distance in metres. Data becomes **information** when it is given context — *Priya's monthly salary is 45,000* — and information becomes **knowledge** when many pieces of it are compared: *salaries in Engineering are 18% above the company average*.

Software exists to turn data into knowledge, and SQL is the language in which that turning is written. Understanding this hierarchy explains why databases are shaped the way they are: a column gives a value its context (`salary`), a row groups related values into one fact about one thing (one employee), and a table collects facts of the same kind so they can be compared.

TERM	MEANING	EXAMPLE
Data	Raw recorded value, no context	45000, 'Priya', 2021-04-01
Information	Data with meaning attached	Priya earns 45000, hired 1 Apr 2021
Knowledge	Patterns derived from information	Staff hired in 2021 earn 12% less than 2019 hires
Metadata	Data describing data	salary is DECIMAL(10,2), not null

1.2 What is a database?

A **database** is an organised collection of related data, stored so that it can be retrieved, updated and protected efficiently. The word "organised" is doing all the work. A folder of spreadsheets is a collection of data; it is not a database, because nothing enforces that the customer id in one file matches the customer id in another, nothing prevents two people saving over each other's edits, and nothing lets you ask a question across all the files in one step.

A real database gives you five things a pile of files cannot:

- **Structure** — every record of a kind has the same shape, declared in advance.
- **Integrity** — rules ("an order must belong to a real customer") are enforced by the database, not hoped for by the application.

- **Concurrency** — thousands of users read and write at once without corrupting each other's work.
- **Durability** — once the database says "saved", the data survives a power cut.
- **Querying** — you describe the answer you want and the system works out how to get it.

1.3 What is a DBMS?

A **Database Management System** is the software that stores the database and mediates all access to it. You never touch the data files yourself; you ask the DBMS. It is responsible for storage layout, caching, query parsing and planning, permission checks, transaction control, crash recovery, backups and replication.

Historically several DBMS models existed, and interviewers still ask about them:

MODEL	HOW DATA IS ORGANISED	EXAMPLE
Hierarchical	A tree; each child has one parent	IBM IMS
Network	A graph; a child may have many parents	IDMS
Relational	Tables of rows and columns, linked by values	MySQL, PostgreSQL, Oracle
Object-oriented	Objects with identity and methods	db4o, ObjectDB
Document / key-value / graph	Flexible documents, pairs, or nodes and edges	MongoDB, Redis, Neo4j

1.4 What is an RDBMS?

An **RDBMS** is a DBMS built on the *relational model*, published by E. F. Codd in 1970. Its central idea is disarmingly simple: store all data in tables (mathematically, *relations*), and express relationships between tables not with pointers but with shared *values*. If an order needs to know its customer, it stores the customer's id. Nothing is nested; nothing is hidden.

From that one decision follow all the properties you rely on: any table can be joined to any other on any matching values; new relationships can be added without rewriting existing data; and the whole database can be queried with one small algebra of operations (select, project, join, union) rather than a bespoke traversal per question.

INTERVIEW TIP

"Is MySQL a DBMS or an RDBMS?" It is both — every RDBMS is a DBMS. The distinction interviewers want: an RDBMS enforces the relational model, supports keys and referential integrity, and is queried with SQL; a plain DBMS need not.

1.5 What is SQL?

SQL (Structured Query Language, pronounced "sequel" or "ess-cue-el") is the standard language for defining, manipulating and querying relational data. It was created at IBM in the 1970s as SEQUEL and standardised by ANSI in 1986; the current standard is SQL:2023.

SQL is **declarative**. You do not write loops that walk the data; you state the properties of the result you want, and the database's optimizer decides how to compute it. This single line asks for every employee earning above 50,000, sorted:

```
SELECT name, salary
FROM employees
WHERE salary > 50000
ORDER BY salary DESC;
```

Nothing in that statement says whether to scan the table, use an index, sort in memory or on disk, or use eight CPU threads. Those are the optimizer's choices, and they may differ tomorrow when the table is larger. That is the source of SQL's power and of its performance surprises — both of which this book covers.

1.6 SQL vs MySQL

This confusion is the single most common beginner error, and it is asked in almost every fresher interview.

	SQL	MYSQL
What it is	A language	A software product (an RDBMS)
Who defines it	ANSI / ISO standard	Oracle Corporation (originally MySQL AB)
Can you install it	No — you cannot install a language	Yes
Versions	SQL-92, SQL:1999 … SQL:2023	5.7, 8.0, 8.4 …
Relationship	MySQL implements most of SQL, plus extensions	One of many implementations

Other implementations you will meet: **PostgreSQL** (open source, strictest standard compliance, rich types), **Oracle Database** (enterprise, PL/SQL), **Microsoft SQL Server** (T-SQL, Windows-first heritage), **SQLite** (embedded, single file, in your phone), **MariaDB** (a MySQL fork).

1.7 DBMS vs RDBMS

ASPECT	DBMS	RDBMS
Data storage	Files, trees, objects — any structure	Tables of rows and columns
Relationships	Pointers, nesting, or none	Foreign keys on shared values
Normalization	Not supported as a concept	Central to design
Query language	Product-specific API	SQL
Multi-user access	Limited	Full concurrency with transactions
Integrity constraints	Usually in the application	Declared in the schema, enforced by the engine
Data volume	Small to moderate	Small to very large
Examples	MS Access (early), file systems, XML stores	MySQL, PostgreSQL, Oracle, SQL Server

1.8 SQL vs NoSQL

"NoSQL" does not mean "no SQL" so much as "not only SQL": a family of databases that relax the relational model in exchange for flexibility or horizontal scale. They are not a replacement for relational databases; they are a different trade-off.

ASPECT	SQL (RELATIONAL)	NOSQL
Schema	Fixed, declared before insert	Flexible or schema-less
Structure	Tables, rows, columns	Documents, key-value, wide-column, graph
Joins	First-class	Usually absent; data is duplicated instead
Transactions	ACID across many tables	Often limited to one document; growing support
Scaling	Vertical first; sharding is work	Horizontal by design
Query language	SQL, standardised	Product-specific APIs
Best for	Money, inventory, bookings, anything with invariants	Event logs, catalogues with varying attributes, caches, social graphs
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Cassandra, Redis, DynamoDB, Neo4j

INTERVIEW TIP

When asked "which would you choose?", never answer with a preference. Answer with a question about the data: does it have invariants that must never break (a bank balance), and do queries cross entity boundaries? Then relational. Is the shape of each record unpredictable and are reads by a single key? Then a document store is reasonable.

1.9 Database vs table, rows and columns, records and fields

A **database** (in MySQL, a synonym for *schema*) is a named container. Inside it live **tables**, and also views, procedures, functions, triggers and indexes. A table is a two-dimensional grid of one kind of thing.

FORMAL TERM	EVERYDAY TERM	MEANING
Relation	Table	All the facts of one kind
Tuple	Row / record	One complete fact: one employee
Attribute	Column / field	One property of that fact: salary
Degree	Column count	How many attributes the table has
Cardinality	Row count	How many records the table holds
Domain	Data type + constraints	The set of legal values for a column

Strictly, a *record* is the stored row and a *field* is the single cell where a row meets a column; in daily speech row/record and column/field are used interchangeably. Two properties matter more than the naming: rows in a true relation have **no inherent order**, and each cell holds **exactly one value** (this is first normal form, Part 21).

```
employees                <- table (relation)
+--------+--------+---------+--------+
| emp_id | name   | dept_id | salary |   <- columns (attributes)
+--------+--------+---------+--------+
|      1 | Priya  |      10 |  72000 |   <- row (tuple / record)
|      2 | Arjun  |      20 |  54000 |
|      3 | Meera  |      10 |  91000 |
+--------+--------+---------+--------+
        ^
        one field (cell)
```

1.10 Schema

The **schema** is the structure of the database: which tables exist, which columns they have, of what types, with what keys and constraints. It is the design, not the data. Three senses of the word appear in practice, and mixing them up costs marks:

- **Logical schema** — tables, columns, keys, relationships. What we mean when we say "design the schema".
- **Physical schema** — how it is stored: files, pages, indexes, partitions.
- **Namespace** — in MySQL, `SCHEMA` is literally a synonym for `DATABASE`. In PostgreSQL and Oracle they are different: one database contains many schemas.

The related term **instance** means the data present at a moment. Schema changes rarely; the instance changes every second.

1.11 Relationships

Data is interesting because it is connected. Relational databases express connection by putting the key of one table into another. Three shapes cover everything, and Part 20 designs with them properly.

RELATIONSHIP	MEANING	HOW IT IS STORED	EXAMPLE
One-to-one	Each row on one side matches at most one on the other	Foreign key with a UNIQUE constraint, or a shared primary key	employee ↔ passport
One-to-many	One row relates to many rows	Foreign key on the "many" side	department → employees
Many-to-many	Rows on both sides relate to many	A third junction table holding both keys	students ↔ courses

A special case worth naming early: a **self-relationship**, where a table references itself. An employee's manager is another employee, so `employees.manager_id` is a foreign key to `employees.emp_id`. Part 14 queries it with a self-join and Part 17 with a recursive CTE.

1.12 The principles of the relational model

Six ideas underpin everything else in this book:

1. **Everything is a table.** Query results are tables too, which is why a query can feed another query (Part 16).
2. **Each row is uniquely identifiable** by a primary key. Without one, you cannot reliably update or delete a specific row.
3. **Order is not information.** If you want an order, you must ask for it with `ORDER BY`.
4. **Relationships live in values**, not in physical links, so any two tables can be joined at query time.
5. **Integrity is declared.** Constraints in the schema outlive every application that talks to the database.
6. **Access is set-based.** Statements act on whole sets of rows at once, not row by row — the mental shift that separates beginners from competent SQL writers.

COMMON MISTAKE

Thinking in loops. Beginners coming from Python or Java try to "go through each row and check". In SQL you describe the whole set: `UPDATE employees SET salary = salary * 1.1 WHERE dept_id = 10;` raises every Engineering salary in one statement. Row-by-row thinking produces slow queries and cursor-shaped code.

1.13 Advantages and limitations of databases

ADVANTAGES	LIMITATIONS
Redundancy is controlled, so facts are stored once	Licensing, hardware and hosting cost money
Integrity rules are enforced centrally	Schema changes on huge tables are slow and risky
Many users work concurrently and safely	Requires skilled design; a bad schema is expensive to fix
Security is granular, down to column level	A single database is a single point of failure without replication
Backup and point-in-time recovery are built in	Horizontal scaling of writes is genuinely hard
One declarative language answers new questions	Rigid structure fits poorly when every record differs

1.14 Real-world database examples

SYSTEM	WHAT THE DATABASE HOLDS	CHARACTERISTIC QUERY
Banking	Accounts, transactions, ledgers, KYC	Transfer money atomically between two accounts
E-commerce	Products, carts, orders, payments, returns	Revenue per category this month vs last
Food delivery	Restaurants, menus, riders, live orders	Average delivery time by area and hour
Hospital	Patients, doctors, appointments, prescriptions	Which doctor has free slots on Friday
College	Students, courses, enrolments, marks	Rank students per course by marks
Ride hailing	Drivers, trips, fares, ratings	Surge pricing input: trips per zone per minute
Streaming	Titles, viewing events, subscriptions	Retention of users who joined in March

CHAPTER SUMMARY

- Data is a raw fact; context makes it information; comparison makes it knowledge.
- A database is organised, protected, concurrently accessible data; a DBMS is the software managing it; an RDBMS is a DBMS following the relational model.
- SQL is a declarative standard language; MySQL is one product that implements it.
- Table = relation, row = tuple = record, column = attribute = field.
- Relationships are stored as shared values: one-to-one, one-to-many, many-to-many via a junction table.
- Rows have no order; every row should be identifiable by a primary key; think in sets, never in loops.

INTERVIEW QUESTIONS — PART ONE

1. **Difference between DBMS and RDBMS?** Beginner An RDBMS stores data in related tables and enforces the relational model with keys, integrity constraints and SQL; a DBMS need not.
2. **Is SQL a programming language?** Beginner It is a declarative, domain-specific language; it is Turing-complete only with procedural extensions (PL/SQL, T-SQL, MySQL routines).
3. **What is the difference between a schema and an instance?** Beginner Schema is the structure and changes rarely; instance is the data at a point in time.
4. **Why is the relational model called "relational"?** Intermediate From the mathematical *relation* (a set of tuples), not from relationships between tables — a favourite trick question.
5. **Can a table exist without a primary key?** Intermediate Physically yes; logically it is a bad design, because duplicate rows become indistinguishable and replication and updates become unsafe. InnoDB will silently create a hidden clustered index.
6. **When would you not use a relational database?** Advanced Very high write throughput with no cross-entity invariants, wildly heterogeneous records, or pure key-based access at massive scale.

PRACTICE — PART ONE

1. Write down, in plain English, the tables you would need for a college library. Beginner
2. For each pair, state the relationship type: author/book, student/roll number, doctor/patient, order/order line. Beginner
3. List three integrity rules a food-delivery database must enforce that an application alone could not guarantee. Intermediate
4. Take a spreadsheet you use and identify what breaks when two people edit it at once. Map each break to one of the five properties in section 1.2. Intermediate

PART TWO

Setting up SQL

A working install, a database of your own, and the first table — with the errors you will hit on the way explained before you hit them.

2.1 What you are installing

MySQL ships as three separable pieces, and knowing which is which prevents most setup confusion:

PIECE	NAME	ROLE
Server	mysqld	The database engine. A background service that owns the data files and listens on port 3306.
Client (CLI)	mysql	A terminal program that sends your SQL to the server and prints results.
Client (GUI)	MySQL Workbench	A desktop application: query editor, results grid, schema designer, admin panels.

Everything you type goes client → server → client. If the server is not running, no client can help you; that is the meaning of the classic *Can't connect to local MySQL server* error.

2.2 Installing MySQL on Windows

1. Download the **MySQL Installer for Windows** from dev.mysql.com/downloads/installer.
2. Choose setup type **Developer Default** — this installs server, Workbench, the CLI and sample data.
3. At *Type and Networking*, keep *Development Computer* and port **3306**.
4. At *Authentication Method*, keep **Use Strong Password Encryption** (caching_sha2_password). Choose the legacy option only if an old application driver requires it.
5. Set the **root password**. Write it down. Recovering it means restarting the server with `--skip-grant-tables`.
6. At *Windows Service*, keep *Configure MySQL Server as a Windows Service* and *Start at System Startup* ticked.
7. Finish, then confirm the service is running: Win+R → `services.msc` → look for *MySQL80*, status *Running*.

2.3 Installing on macOS and Linux

```
# macOS (Homebrew)
brew install mysql
brew services start mysql
mysql_secure_installation      # sets the root password, removes test db

# Ubuntu / Debian
sudo apt update
sudo apt install mysql-server
sudo systemctl enable --now mysql
sudo mysql_secure_installation

# Verify on any platform
mysql --version
mysqladmin -u root -p status
```

On a fresh Ubuntu install, root authenticates through the operating system socket, so `sudo mysql` works without a password while `mysql -u root -p` fails. Create a normal user for daily work instead of fighting it:

```
CREATE USER 'learner'@'localhost' IDENTIFIED BY 'Str0ng#Pass';
GRANT ALL PRIVILEGES ON company.* TO 'learner'@'localhost';
FLUSH PRIVILEGES;
```

2.4 MySQL Workbench, screen by screen

- **Connection tile** on the home screen — double-click it to open a SQL editor tab against that server.
- **Navigator (left)** — Schemas lists your databases; expand to Tables, Views, Stored Procedures, Functions.
- **Query editor (centre)** — type SQL here. *Ctrl/Cmd+Enter* runs the statement under the cursor; *Ctrl/Cmd+Shift+Enter* runs the whole script.
- **Result grid (below)** — one tab per result set. The *Form Editor* and *Field Types* side tabs show one row at a time and the column metadata.
- **Action Output (bottom)** — every statement with its duration and row count. Read this, not just the grid; it is where warnings appear.
- **Safe Updates** — Preferences → SQL Editor → "Safe Updates" blocks `UPDATE`/`DELETE` without a key in the WHERE clause. Leave it on; it will save you at least once.

2.5 The command line

```
mysql -u learner -p                 # connect, prompt for password
mysql -u learner -p -h 10.0.0.5 -P 3306 company   # remote host, explicit db

mysql> SHOW DATABASES;              # list databases
mysql> USE company;                 # choose one
mysql> SHOW TABLES;                 # list its tables
mysql> DESCRIBE employees;          # column definitions  (also DESC)
mysql> SHOW CREATE TABLE employees\G  # the exact DDL, vertical output
mysql> SELECT DATABASE(), VERSION(), USER();
mysql> source /path/to/setup.sql;   # run a script file
mysql> \q                           # quit  (also EXIT or QUIT)
```

Two habits from day one: end every statement with a semicolon, and end a statement with `\G` instead when a row is too wide for the terminal — it prints columns vertically.

COMMON MISTAKE

Seeing the prompt change to `->` and panicking. That means MySQL is still waiting for the end of your statement — usually a missing semicolon or an unclosed quote. Type a semicolon, or press *Ctrl+C* to abandon the line.

2.6 Creating and selecting a database

```
CREATE DATABASE IF NOT EXISTS company
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;

USE company;
SELECT DATABASE();   -- confirm: company
```

Always use **utf8mb4**. MySQL's old `utf8` is a three-byte subset that cannot store emoji or some Indic and CJK characters; utf8mb4 is real UTF-8. The collation decides comparison and sorting: `ai_ci` means accent-insensitive, case-insensitive, so *'ARJUN' = 'arjun'* is true. Use `utf8mb4_0900_as_cs` when case must matter.

2.7 The book's setup script

Run this once. Every example in Volumes I and II uses it.

```
DROP DATABASE IF EXISTS company;
CREATE DATABASE company CHARACTER SET utf8mb4;
USE company;
```

```
CREATE TABLE departments (
  dept_id    INT PRIMARY KEY,
  dept_name  VARCHAR(50) NOT NULL UNIQUE,
  location   VARCHAR(50)
);

CREATE TABLE employees (
  emp_id      INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(60) NOT NULL,
  email       VARCHAR(120) UNIQUE,
  dept_id     INT,
  manager_id  INT,
  salary      DECIMAL(10,2) NOT NULL CHECK (salary > 0),
  hire_date   DATE NOT NULL,
  city        VARCHAR(40),
  CONSTRAINT fk_emp_dept    FOREIGN KEY (dept_id)    REFERENCES
departments(dept_id),
  CONSTRAINT fk_emp_manager FOREIGN KEY (manager_id) REFERENCES employees(emp_id)
);

CREATE TABLE customers (
  cust_id     INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(60) NOT NULL,
  city        VARCHAR(40),
  signup_date DATE NOT NULL
);

CREATE TABLE products (
  product_id   INT AUTO_INCREMENT PRIMARY KEY,
  product_name VARCHAR(80) NOT NULL,
  category     VARCHAR(40) NOT NULL,
  price        DECIMAL(10,2) NOT NULL
);

CREATE TABLE orders (
  order_id   INT AUTO_INCREMENT PRIMARY KEY,
  cust_id    INT NOT NULL,
  order_date DATE NOT NULL,
  status     ENUM('PLACED','SHIPPED','DELIVERED','CANCELLED') DEFAULT 'PLACED',
  amount     DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (cust_id) REFERENCES customers(cust_id)
);

CREATE TABLE order_items (
  order_id   INT NOT NULL,
  product_id INT NOT NULL,
  qty        INT NOT NULL,
  unit_price DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (order_id, product_id),
  FOREIGN KEY (order_id)   REFERENCES orders(order_id) ON DELETE CASCADE,
  FOREIGN KEY (product_id) REFERENCES products(product_id)
```

```
);

INSERT INTO departments VALUES
 (10,'Engineering','Bengaluru'), (20,'Sales','Mumbai'),
 (30,'Finance','Pune'),          (40,'HR','Bengaluru'),
 (50,'Research',NULL);

INSERT INTO employees (name,email,dept_id,manager_id,salary,hire_date,city) VALUES
 ('Priya Nair','priya@corp.in',10,NULL,145000,'2016-03-01','Bengaluru'),
 ('Arjun Rao','arjun@corp.in',10,1,98000,'2018-07-15','Bengaluru'),
 ('Meera Iyer','meera@corp.in',10,1,91000,'2019-01-10','Hyderabad'),
 ('Kabir Shah','kabir@corp.in',20,NULL,120000,'2017-05-20','Mumbai'),
 ('Sana Qureshi','sana@corp.in',20,4,76000,'2020-02-01','Mumbai'),
 ('Dev Patel','dev@corp.in',20,4,68000,'2021-09-09','Ahmedabad'),
 ('Riya Sen','riya@corp.in',30,NULL,110000,'2015-11-11','Pune'),
 ('Vikram Das','vikram@corp.in',30,7,64000,'2022-06-01','Pune'),
 ('Nisha Gupta','nisha@corp.in',40,NULL,72000,'2019-08-19','Bengaluru'),
 ('Omar Farooq',NULL,NULL,NULL,55000,'2023-01-03','Delhi');

INSERT INTO customers (name,city,signup_date) VALUES
 ('Anita Desai','Bengaluru','2022-01-14'), ('Rohan Mehta','Mumbai','2022-03-02'),
 ('Farid Khan','Delhi','2022-06-21'),      ('Leela Menon','Kochi','2023-02-08'),
 ('Tarun Bose','Kolkata','2023-05-30'),    ('Ishita Roy','Mumbai','2023-11-17');

INSERT INTO products (product_name,category,price) VALUES
 ('Mechanical Keyboard','Electronics',4500),('27in Monitor','Electronics',18000),
 ('Office Chair','Furniture',9500),        ('Standing Desk','Furniture',22000),
 ('Noise Cancelling Headset','Electronics',7800),
 ('Desk Lamp','Furniture',1200),           ('Webcam 1080p','Electronics',3200);

INSERT INTO orders (cust_id,order_date,status,amount) VALUES
 (1,'2023-01-10','DELIVERED',22500),(1,'2023-04-02','DELIVERED',9500),
 (2,'2023-02-18','SHIPPED',18000),  (3,'2023-03-05','CANCELLED',1200),
 (1,'2023-08-21','DELIVERED',3200), (4,'2023-09-14','DELIVERED',29800),
 (2,'2024-01-07','PLACED',7800),    (5,'2024-02-29','DELIVERED',22000);

INSERT INTO order_items VALUES
 (1,1,1,4500),(1,2,1,18000),(2,3,1,9500),(3,2,1,18000),(4,6,1,1200),
 (5,7,1,3200),(6,4,1,22000),(6,5,1,7800),(7,5,1,7800),(8,4,1,22000);
```

Note the deliberate awkwardness in this data: Omar has no department, no manager and no email; department 50 has no employees; customer 6 has never ordered; one order is cancelled. Clean data teaches nothing about `NULL`, outer joins or anti-joins — this data does.

2.8 Running your first queries

```
SELECT * FROM departments;
SELECT name, salary FROM employees WHERE dept_id = 10;
SELECT COUNT(*) AS headcount FROM employees;
```

```
+-----------+--------+
| name      | salary |
+-----------+--------+
| Priya Nair|145000.00|
| Arjun Rao | 98000.00|
| Meera Iyer| 91000.00|
+-----------+--------+
3 rows in set (0.00 sec)
```

2.9 Understanding SQL errors

ERROR	MEANING	FIX
1046 — No database selected	You ran a statement without choosing a database	`USE company;` or qualify names: `company.employees`
1146 — Table doesn't exist	Wrong name, wrong database, or wrong case on Linux	`SHOW TABLES;` and check spelling and case
1054 — Unknown column	Typo, or the column belongs to a table not in FROM	`DESC table;`
1064 — Syntax error near '…'	The parser failed at the quoted text; the real mistake is usually just before it	Check commas, quotes, reserved words
1062 — Duplicate entry for key	You violated PRIMARY KEY or UNIQUE	Use a different value, or `INSERT … ON DUPLICATE KEY UPDATE`
1452 — Cannot add or update a child row	Foreign key points at a parent that does not exist	Insert the parent row first
1451 — Cannot delete or update a parent row	Children still reference it	Delete children, or define ON DELETE CASCADE
1364 — Field has no default value	NOT NULL column omitted from INSERT	Supply a value or add a DEFAULT
1055 — Not in GROUP BY (only_full_group_by)	You selected a non-aggregated column not in GROUP BY	Group by it or aggregate it (Part 13)
1175 — Safe update mode	Workbench blocked a keyless UPDATE/DELETE	Add a key to WHERE, or `SET SQL_SAFE_UPDATES=0;`
2002 / 2003 — Can't connect	Server is not running or wrong host/port	Start the service; check port 3306 and firewall
1045 — Access denied for user	Wrong password, or user not granted from this host	Check the user@host pair and its grants

Read errors from the number, not the prose: the number identifies the class of problem exactly. And always run `SHOW WARNINGS;` after a statement that "worked but looks wrong" — truncated strings and out-of-range numbers are reported there, not as errors.

2.10 SQL comments

```
-- Two dashes and a space: comment to end of line (standard SQL)
# Hash: MySQL-only line comment
/* Block comment,
   spanning lines */
SELECT name /* inline */, salary FROM employees;
/*! MySQL-specific code that other engines skip */
```

The space after `--` is mandatory in MySQL: `--x` is not a comment, and `5--3` evaluates to 8.

2.11 Naming conventions

RULE	DO	AVOID
Case	lower_snake_case	MixedCase, spaces
Table names	Plural or consistent: employees, orders	tbl_Employee_Data_New
Primary keys	emp_id, order_id	bare "id" everywhere, making joins ambiguous
Foreign keys	Same name as the parent key	emp, employee_fk_1
Booleans	is_active, has_gst	flag, status2
Dates	created_at, hire_date	date, dt
Reserved words	order_date, grp	order, group, desc, rank — needing backticks forever
Constraints	pk_, fk_, uq_, chk_ prefixes	engine-generated names you cannot remember

Identifiers may be quoted with backticks in MySQL (`` `order` ``), double quotes in PostgreSQL, Oracle and standard SQL, and square brackets in SQL Server (`[order]`). Needing them is a smell: rename the column instead.

IMPORTANT — CASE SENSITIVITY

Keywords are never case-sensitive. *Table names are*, on Linux and macOS with case-sensitive volumes, because MySQL maps each table to a file. `Employees` and `employees` are the same table on Windows and different tables on Linux. Write everything lowercase and the problem never occurs.

CHAPTER SUMMARY

- Server (mysqld), CLI client (mysql) and Workbench are three separate things; connection errors are nearly always a stopped server or a wrong user@host.
- Create databases as utf8mb4; pick the collation deliberately for case sensitivity.
- `SHOW DATABASES / TABLES`, `DESC t` and `SHOW CREATE TABLE t` are your orientation tools.
- Error numbers, not error prose, identify the problem; `SHOW WARNINGS` catches silent damage.
- Name things in lower_snake_case, avoid reserved words, and keep Safe Updates on.

INTERVIEW QUESTIONS — PART TWO

1. **Default MySQL port?** Beginner 3306 (X Protocol 33060). PostgreSQL 5432, SQL Server 1433, Oracle 1521.
2. **Difference between utf8 and utf8mb4 in MySQL?** Intermediate utf8 stores at most 3 bytes per character and cannot hold emoji or all supplementary-plane characters; utf8mb4 is true 4-byte UTF-8 and is the default in MySQL 8.
3. **What does a collation control?** Intermediate Comparison and sort order, including case and accent sensitivity — and therefore which values a UNIQUE index treats as duplicates.
4. **How do you see the exact definition of an existing table?** Beginner `SHOW CREATE TABLE t;` — more complete than DESCRIBE, which omits keys and options.
5. **Why might a query work on your laptop and fail on the Linux server?** Advanced Table-name case sensitivity (`lower_case_table_names`), or different `sql_mode` — typically `only_full_group_by` or `strict_trans_tables`.

PRACTICE — PART TWO

1. Install MySQL, run the setup script, and confirm all six tables exist. Beginner
2. Deliberately cause errors 1146, 1062 and 1452, and read each message. Beginner
3. Create a second database `company_cs` with a case-sensitive collation and show that `'a' = 'A'` differs between the two. Intermediate
4. Create the `learner` user with rights on `company` only, connect as them, and confirm they cannot see other databases. Intermediate

PART THREE

SQL commands

Five families, sixteen statements. Learn the families first and every SQL statement you ever meet has an obvious home.

3.1 The five command families

FAMILY	FULL NAME	ACTS ON	STATEMENTS	AUTO-COMMIT?
DDL	Data Definition Language	Structure	CREATE, ALTER, DROP, TRUNCATE, RENAME	Yes — implicit commit
DML	Data Manipulation Language	Rows	INSERT, UPDATE, DELETE	No — transactional
DQL	Data Query Language	Reading	SELECT	n/a
DCL	Data Control Language	Permissions	GRANT, REVOKE	Yes
TCL	Transaction Control Language	Transactions	COMMIT, ROLLBACK, SAVEPOINT, SET TRANSACTION	n/a

The auto-commit column is the one interviewers probe. In MySQL, DDL causes an **implicit commit**: you cannot wrap `ALTER TABLE` in a transaction and roll it back. PostgreSQL and SQL Server *do* support transactional DDL — a genuine dialect difference worth stating in an interview.

3.2 DDL — CREATE

Definition. Creates a new database object: a database, table, index, view, procedure, function, trigger or user.

```
CREATE TABLE [IF NOT EXISTS] table_name (
    column_name datatype [column_constraints],
    ...
    [table_constraints]
) [ENGINE=InnoDB] [DEFAULT CHARSET=utf8mb4];
```

Example.

```
CREATE TABLE projects (
  project_id  INT AUTO_INCREMENT PRIMARY KEY,
  title       VARCHAR(100) NOT NULL,
  dept_id     INT NOT NULL,
  budget      DECIMAL(12,2) DEFAULT 0,
  started_on  DATE,
  FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
) ENGINE=InnoDB;
```

Result. `Query OK, 0 rows affected`. The table exists and is empty.

Real-world use. Every schema migration begins here. In production, CREATE statements live in versioned migration files (Flyway, Liquibase, Alembic, Rails), never typed by hand into a console.

Common mistakes. Forgetting the engine on old servers (MyISAM silently ignores foreign keys); creating a table without a primary key; using `VARCHAR(255)` for everything; and putting the foreign key before the parent table exists.

Interview question. *Can CREATE TABLE be rolled back in MySQL?* No — DDL implicitly commits the open transaction. Intermediate

CREATE TABLE ... AS SELECT (CTAS) and LIKE

```
-- Structure + data, but NOT keys, indexes or constraints
CREATE TABLE emp_backup AS SELECT * FROM employees;

-- Structure only, WITH keys and indexes, no rows
CREATE TABLE emp_clone LIKE employees;
INSERT INTO emp_clone SELECT * FROM employees;   -- then copy rows
```

This distinction is a frequent interview question: CTAS copies data and column types but loses the primary key, indexes and auto-increment; `LIKE` copies the full definition but no data.

3.3 DDL — ALTER

Definition. Changes the structure of an existing object without dropping it.

```
ALTER TABLE employees ADD COLUMN phone VARCHAR(15);
ALTER TABLE employees ADD COLUMN grade CHAR(1) AFTER salary;
ALTER TABLE employees MODIFY COLUMN name VARCHAR(80) NOT NULL;
ALTER TABLE employees CHANGE COLUMN phone mobile VARCHAR(15);
ALTER TABLE employees RENAME COLUMN city TO base_city;   -- MySQL 8+
ALTER TABLE employees DROP COLUMN grade;
ALTER TABLE employees ADD CONSTRAINT uq_email UNIQUE (email);
ALTER TABLE employees ADD INDEX idx_dept (dept_id);
```

```
ALTER TABLE employees DROP FOREIGN KEY fk_emp_dept;
ALTER TABLE employees AUTO_INCREMENT = 1000;
```

CLAUSE	CHANGES	NEEDS FULL DEFINITION RESTATED?
MODIFY	Type, nullability, default — same name	Yes
CHANGE	Name *and* definition	Yes
RENAME COLUMN	Name only (MySQL 8+)	No
ALTER COLUMN … SET DEFAULT	Default only	No

Real-world use. Adding a column to a 200-million-row table is an operations event, not a keystroke. MySQL 8 performs many changes *in place, instantly* (`ALGORITHM=INSTANT` for adding a trailing column); others rebuild the whole table and lock writes. Teams use `pt-online-schema-change` or `gh-ost` for the rest.

Common mistakes. Using MODIFY and forgetting `NOT NULL` or the default, silently dropping the constraint; narrowing a column (`VARCHAR(100)→VARCHAR(20)`) and truncating data; adding a NOT NULL column with no default to a non-empty table.

Interview question. *Difference between MODIFY and CHANGE?* CHANGE can also rename the column and therefore requires both the old and the new name. Beginner

3.4 DDL — DROP, TRUNCATE, RENAME

```
DROP TABLE IF EXISTS emp_backup;      -- object and data gone
DROP DATABASE IF EXISTS scratch;
TRUNCATE TABLE audit_log;             -- all rows gone, table remains
RENAME TABLE employees TO staff;      -- MySQL
ALTER TABLE employees RENAME TO staff;-- also valid, and the standard form
```

	DELETE	TRUNCATE	DROP
Family	DML	DDL	DDL
Removes	Selected rows	All rows	Rows + structure
WHERE clause	Yes	No	No
Rollback	Yes, inside a transaction	No (implicit commit)	No
Triggers fire	Yes	No	No
Speed on big tables	Slow, row by row, logged	Very fast — drops and recreates storage	Very fast
AUTO_INCREMENT	Preserved	Reset to 1	Gone with the table
Rows reported	Actual count	0	0
Foreign keys	Respected / cascades	Refused if a FK references the table	Refused unless children dropped or FKs off

INTERVIEW TIP

This table is the most-asked comparison in SQL interviews. If you can recite the rollback, trigger, auto-increment and rows-reported rows without hesitating, you will pass that question — and the follow-up: *"why is TRUNCATE faster?"* Because it deallocates the data pages wholesale instead of logging one delete per row.

3.5 DML — INSERT, UPDATE, DELETE at a glance

Part 7 treats these in depth; here is the family view, with the properties that make them DML rather than DDL.

```
INSERT INTO departments (dept_id, dept_name, location)
VALUES (60, 'Legal', 'Delhi');

UPDATE employees
   SET salary = salary * 1.10
 WHERE dept_id = 10;

DELETE FROM orders
 WHERE status = 'CANCELLED' AND order_date < '2023-01-01';
```

Every DML statement is transactional. With `autocommit=0` or inside `START TRANSACTION`, nothing is permanent until `COMMIT`; `ROLLBACK` undoes it. This is your safety net for the mistake in the next box.

COMMON MISTAKE

`UPDATE employees SET salary = 50000;` — no WHERE, so every employee now earns 50,000. The habit that prevents it: write the statement as a `SELECT` with the same WHERE first, check the rows, then convert it. Keep Safe Updates on, and run destructive statements inside an explicit transaction.

3.6 DQL — SELECT

Definition. Retrieves rows. It is the only DQL statement, and it is where 90% of professional SQL time is spent. Its full clause order:

```
SELECT   [DISTINCT] select_list
FROM     table [join_clauses]
WHERE    row_conditions
GROUP BY grouping_columns
HAVING   group_conditions
ORDER BY sort_columns [ASC|DESC]
LIMIT    n [OFFSET m];
```

Logical execution order differs from written order, and this single fact explains most beginner confusion: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. Because SELECT runs after WHERE, a column alias defined in SELECT cannot be used in WHERE — but it can be used in ORDER BY, which runs later. Part 8 returns to this with examples.

Interview question. *Why can't I use a SELECT alias in WHERE?* Because WHERE is evaluated before the select list exists. Intermediate

3.7 DCL — GRANT and REVOKE

```
GRANT SELECT, INSERT ON company.orders TO 'app_user'@'%';
GRANT SELECT (name, salary) ON company.employees TO 'audit'@'localhost';
GRANT ALL PRIVILEGES ON company.* TO 'dba'@'localhost' WITH GRANT OPTION;
REVOKE INSERT ON company.orders FROM 'app_user'@'%';
SHOW GRANTS FOR 'app_user'@'%';
FLUSH PRIVILEGES;   -- needed only after editing grant tables directly
```

Real-world use. The application account should hold SELECT, INSERT, UPDATE and DELETE on its own schema — and nothing else. No DROP, no GRANT OPTION, no access to other schemas. Analysts get a read-only account; migrations run under a separate account used only by the deployment pipeline. This is *least privilege*, and Part 32 develops it.

Common mistakes. Granting on `*.*` when you meant one schema; using `'user'@'%'` and wondering why `'user'@'localhost'` has different rights (MySQL identifies a user by the pair); forgetting that revoking a privilege does not disconnect existing sessions.

Interview question. *Is GRANT DDL or DCL?* DCL. Only permission statements are DCL. Beginner

3.8 TCL — COMMIT, ROLLBACK, SAVEPOINT

```
START TRANSACTION;                      -- or BEGIN
  UPDATE accounts SET balance = balance - 5000 WHERE acc_id = 1;
  SAVEPOINT after_debit;
  UPDATE accounts SET balance = balance + 5000 WHERE acc_id = 2;
  -- something is wrong with the credit only:
  ROLLBACK TO SAVEPOINT after_debit;    -- debit survives, credit undone
  UPDATE accounts SET balance = balance + 5000 WHERE acc_id = 9;
COMMIT;                                 -- both changes become permanent

RELEASE SAVEPOINT after_debit;           -- discard the marker
SET autocommit = 0;                      -- every statement then needs COMMIT
```

STATEMENT	EFFECT
START TRANSACTION / BEGIN	Opens a transaction; suspends autocommit for its duration
COMMIT	Makes all changes since the start permanent and visible to others
ROLLBACK	Discards all changes since the start
SAVEPOINT name	Marks a point you can partially roll back to
ROLLBACK TO SAVEPOINT name	Undoes work after the marker; the transaction stays open
SET TRANSACTION ISOLATION LEVEL …	Chooses the concurrency guarantees (Part 26)

Real-world use. A money transfer must debit one account and credit another or do neither. That "or neither" is the entire reason transactions exist, and Part 25 treats ACID properly.

Common mistakes. Assuming a rollback can undo a `TRUNCATE` or an `ALTER` (it cannot — DDL commits implicitly); leaving a transaction open, holding locks and blocking other users; assuming MyISAM tables are transactional (they are not).

Interview question. *What happens to an open transaction if the client disconnects?* The server rolls it back. Intermediate

QUICK REVISION — PART THREE

```
DDL  structure   CREATE ALTER DROP TRUNCATE RENAME      auto-commits
DML  rows        INSERT UPDATE DELETE                   transactional
DQL  reading     SELECT
DCL  rights      GRANT REVOKE
TCL  atomicity   COMMIT ROLLBACK SAVEPOINT

DELETE  = DML, WHERE, rollback yes, triggers yes, AI kept
TRUNCATE= DDL, all rows, rollback no,  triggers no,  AI reset
DROP    = DDL, object gone
```

INTERVIEW QUESTIONS — PART THREE

1. **Classify TRUNCATE.** Beginner DDL, despite removing data — it deallocates storage rather than deleting rows.
2. **Which families auto-commit?** Intermediate DDL and DCL. DML does not.
3. **Difference between CTAS and CREATE TABLE LIKE?** Intermediate CTAS copies data but not keys/indexes/auto-increment; LIKE copies the definition but no data.
4. **Can you roll back a DROP TABLE in MySQL? In PostgreSQL?** Advanced No in MySQL; yes in PostgreSQL, which has transactional DDL.
5. **Why can TRUNCATE fail on a table with no rows?** Advanced Because another table's foreign key references it; MySQL refuses TRUNCATE on a referenced parent.
6. **What does WITH GRANT OPTION do?** Intermediate Lets the grantee pass the same privileges to other users — rarely appropriate for application accounts.

PRACTICE — PART THREE

1. Create the `projects` table from 3.2, then add a `status` column with a default, rename it, and drop it. Beginner
2. Make both copies of `employees` (CTAS and LIKE) and use `SHOW CREATE TABLE` to prove what each lost. Intermediate
3. Inside a transaction, delete all rows from `emp_backup`, check the count, then roll back. Repeat with TRUNCATE and explain the result. Intermediate
4. Grant an `analyst` user SELECT on two columns of `employees` only, and verify that `SELECT *` fails for them. Advanced

PART FOUR

Data types

The type you choose is a constraint, a storage decision and a performance decision at once. Most schema regrets start here.

4.1 Why types matter

A data type declares the set of legal values for a column — its *domain*. That has four consequences: illegal data is rejected at the door; the engine knows exactly how many bytes each row needs; comparisons and sorting behave correctly (`'10' < '9'` as text but `10 > 9` as numbers); and indexes can be built compactly. A phone number stored as `BIGINT` loses its leading zero; a price stored as `FLOAT` loses money. Both are real production incidents.

4.2 Numeric types

TYPE	BYTES	SIGNED RANGE	UNSIGNED MAX	USE FOR
TINYINT	1	−128 to 127	255	Small enums, flags, ages
SMALLINT	2	−32,768 to 32,767	65,535	Year counts, small lookups
MEDIUMINT	3	±8.3 million	16.7 million	Mid-size id spaces
INT / INTEGER	4	±2.147 billion	4.29 billion	Default choice for ids and counts
BIGINT	8	$\pm 9.22 \times 10^{18}$	1.8×10^{19}	High-volume ids, epoch millis, view counts
DECIMAL(p,s)	varies	Exact, up to 65 digits	—	Money, tax, quantities that must be exact
FLOAT	4	~7 significant digits	—	Sensor readings, approximations
DOUBLE	8	~15 significant digits	—	Scientific values, coordinates
BIT(n)	varies	n bits	—	Packed flags (rarely worth it)

Exact vs approximate: the money rule

```
SELECT 0.1 + 0.2 = 0.3               AS decimal_ok,     -- 1  (exact literals)
       CAST(0.1 AS DOUBLE) + CAST(0.2 AS DOUBLE) = 0.3 AS float_ok;  -- 0

CREATE TABLE invoices (
  amount_good DECIMAL(12,2),   -- 1234567890.12  exact
  amount_bad  DOUBLE           -- 1234567890.1200001  eventually
);
```

`DECIMAL(p,s)` means *p* total digits with *s* after the point: `DECIMAL(10,2)` holds up to 99,999,999.99. Never store currency in FLOAT or DOUBLE; binary floating point cannot represent 0.1 exactly, and errors accumulate over sums. Some teams go further and store money as an integer number of paise or cents in a BIGINT.

COMMON MISTAKE

Believing `INT(11)` limits the value to 11 digits. The number in parentheses is only a *display width* hint, deprecated in MySQL 8.0.17. `INT(1)` and `INT(11)` store exactly the same range. To change the range, change the type.

Dialect note. PostgreSQL: `NUMERIC` is the same as DECIMAL, there is no UNSIGNED, and auto-increment is `GENERATED BY DEFAULT AS IDENTITY` or `SERIAL`. Oracle has a single `NUMBER(p,s)`. SQL Server uses `MONEY` (avoid it; it has only four decimal places) and `IDENTITY(1,1)`.

4.3 String types

TYPE	MAX LENGTH	STORAGE	USE FOR
CHAR(n)	255 chars	Fixed n bytes per row, right-padded	Truly fixed codes: country 'IN', gender 'M', hash of known length
VARCHAR(n)	65,535 bytes per row total	Actual length + 1–2 length bytes	Names, emails, titles — almost all text
TINYTEXT	255 bytes	Off-row possible	Rarely used
TEXT	65,535 bytes (64 KB)	Off-row	Comments, descriptions
MEDIUMTEXT	16 MB	Off-row	Articles, logs
LONGTEXT	4 GB	Off-row	Documents, serialised blobs of text

ASPECT	CHAR	VARCHAR
Length	Fixed	Variable
Padding	Padded with spaces to n; trailing spaces stripped on retrieval	Stored as given
Storage	n bytes always	Length + 1 byte (n≤255) or 2 bytes
Speed	Marginally faster: fixed-width rows	Slightly more work, far less space
Best for	Values of genuinely equal length	Everything else

Two practical rules. First, `VARCHAR(255)` is not a magic number — it is a habit from MySQL 4. Size to the domain: a state code is 2, an email 254, a person's name 80. Oversized VARCHARs cost nothing on disk but inflate temporary tables and index sizes, because those are allocated at maximum width. Second, prefer `VARCHAR` over `TEXT` whenever the value fits: TEXT columns cannot have a default, are stored off-row, and can only be indexed with a prefix length.

```
CREATE TABLE t (c CHAR(5), v VARCHAR(5));
INSERT INTO t VALUES ('ab   ', 'ab   ');
SELECT CONCAT('[',c,']'), CONCAT('[',v,']'), LENGTH(c), LENGTH(v) FROM t;
-- [ab]  [ab   ]  2  5      <- CHAR loses trailing spaces, VARCHAR keeps them
```

Dialect note. PostgreSQL's `TEXT` is unlimited and has no performance penalty — it is the idiomatic choice there. Oracle uses `VARCHAR2(n)` and `CLOB`. SQL Server uses `NVARCHAR(n)` for Unicode and `NVARCHAR(MAX)` for large text.

4.4 Date and time types

TYPE	FORMAT	RANGE	BYTES	TIME ZONE AWARE?
DATE	YYYY-MM-DD	1000-01-01 to 9999-12-31	3	No
TIME	HH:MM:SS	−838:59:59 to 838:59:59	3	No
DATETIME	YYYY-MM-DD HH:MM:SS	1000 to 9999	5–8	No — stored as written
TIMESTAMP	YYYY-MM-DD HH:MM:SS	1970-01-01 to 2038-01-19	4–7	Yes — stored UTC, shown in session zone
YEAR	YYYY	1901 to 2155	1	No

The choice between DATETIME and TIMESTAMP is a real design decision. TIMESTAMP converts to UTC on write and back to the session's time zone on read, which is exactly right for "when did this happen" — but it dies in 2038 and its range starts in 1970, so it cannot hold a date of birth. DATETIME stores the literal you gave with no conversion, which is right for "the meeting is at 9am local" but wrong for events across zones. A common professional pattern: store `DATETIME` in UTC explicitly, and keep the user's zone in a separate column.

```
CREATE TABLE events (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  happened_at TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
  updated_at  TIMESTAMP    DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  meeting_at  DATETIME,                    -- wall-clock, no conversion
  precise_at  DATETIME(6),                 -- microsecond precision
  born_on     DATE
);
SELECT @@time_zone, NOW(), UTC_TIMESTAMP();
```

`ON UPDATE CURRENT_TIMESTAMP` is a MySQL convenience that maintains an *updated_at* column automatically — other engines need a trigger. Fractional seconds go up to `(6)`, microseconds.

Dialect note. PostgreSQL: `TIMESTAMPTZ` is the zone-aware type (and has no 2038 limit); `TIMESTAMP` there means *without* zone. Oracle has `DATE` (which includes a time!) and `TIMESTAMP WITH TIME ZONE`. SQL Server: `DATETIME2` and `DATETIMEOFFSET`; its `TIMESTAMP` is not a date at all but a row-version counter.

4.5 Boolean, ENUM, SET, JSON and binary

TYPE	REALITY IN MYSQL	NOTES
BOOLEAN / BOOL	Synonym for TINYINT(1)	TRUE is 1, FALSE is 0; any non-zero is truthy
ENUM('A','B')	Internally an integer index	Compact and self-documenting, but adding a value needs ALTER TABLE
SET('a','b','c')	Bitmask of up to 64 members	Multiple values in one column — violates 1NF; prefer a child table
JSON	Validated, binary-encoded document	Indexable only through generated columns (Part 31)
BINARY / VARBINARY	Byte strings	Hashes, UUIDs as 16 bytes
BLOB family	TINY/BLOB/MEDIUM/LONG, to 4 GB	Images, files — usually better in object storage with a URL in the table
GEOMETRY, POINT	Spatial types with SRID	Distance and containment queries with spatial indexes

```
-- ENUM: ordering follows declaration order, not alphabet
CREATE TABLE tickets (
  id       INT PRIMARY KEY,
  priority ENUM('LOW','MEDIUM','HIGH','URGENT') NOT NULL DEFAULT 'LOW'
);
SELECT * FROM tickets ORDER BY priority DESC;   -- URGENT first

-- Boolean idiom
ALTER TABLE employees ADD is_active BOOLEAN NOT NULL DEFAULT TRUE;
SELECT COUNT(*) FROM employees WHERE is_active;   -- no "= TRUE" needed
```

When to use ENUM vs a lookup table. ENUM for a short, stable list owned by the schema (order status, priority). A lookup table when the list is data the business edits, needs extra attributes (a display label, a colour, a sort order), or must be referenced by other tables. Interviewers like this question because the wrong answer — "ENUM always, it's faster" — reveals inexperience with migrations.

4.6 Choosing a type: a decision table

YOU ARE STORING	USE	NOT	BECAUSE
Surrogate id	INT / BIGINT UNSIGNED AUTO_INCREMENT	VARCHAR	4–8 bytes, fast joins, compact indexes
Money	DECIMAL(12,2)	FLOAT, DOUBLE	Exactness; rounding errors are unacceptable
Percentage	DECIMAL(5,2)	FLOAT	Exact and bounded
Phone number	VARCHAR(15)	BIGINT	Leading zeros, '+', extensions
PIN / ZIP code	CHAR(6)	INT	Leading zeros are meaningful
Email	VARCHAR(254) UNIQUE	TEXT	Needs a unique index; TEXT indexes need prefixes
Country code	CHAR(2)	VARCHAR(50)	Genuinely fixed length
Yes/no flag	BOOLEAN (TINYINT)	VARCHAR(3) 'Yes'	1 byte, no case or spelling variance
Date of birth	DATE	TIMESTAMP	TIMESTAMP starts at 1970
Event time	TIMESTAMP or UTC DATETIME	VARCHAR	Comparable, sortable, arithmetic works
Free-text description	VARCHAR(1000) or TEXT	CHAR(1000)	Fixed width wastes space per row
Profile photo	VARCHAR URL + object storage	BLOB	Keeps rows small; the database is not a file server
UUID	BINARY(16) (+ UUID_TO_BIN)	CHAR(36)	Less than half the bytes; better index locality

IMPORTANT — STRICT MODE

MySQL 8 runs with `STRICT_TRANS_TABLES` by default, so inserting 'abc' into an INT or a 30-character string into `VARCHAR(10)` raises an error. On older or misconfigured servers the same statements silently store 0 and a truncated string. If you inherit a database, check `SELECT @@sql_mode;` before you trust any data in it.

INTERVIEW QUESTIONS — PART FOUR

1. **CHAR vs VARCHAR?** Beginner Fixed vs variable length; CHAR pads and strips trailing spaces, VARCHAR stores a length prefix and only the bytes used.
2. **Why not FLOAT for money?** Beginner Binary floating point cannot represent decimal fractions exactly; errors accumulate. Use DECIMAL.
3. **DATETIME vs TIMESTAMP?** Intermediate TIMESTAMP is UTC-converted, 4 bytes, 1970–2038; DATETIME stores the literal, 1000–9999. Choose by whether the instant or the wall-clock time is the fact.
4. **What does the number in INT(11) mean?** Intermediate Display width only, now deprecated — not a range limit.
5. **How would you store a UUID efficiently in MySQL?** Advanced `BINARY(16)` with `UUID_TO_BIN(uuid, 1)` to swap the time bytes for index locality.
6. **Downside of ENUM?** Advanced Adding or reordering values requires ALTER TABLE; it sorts by internal index, not alphabetically; and the list is invisible to other tables.
7. **What is the 65,535-byte limit?** Advanced A whole InnoDB row (excluding off-row TEXT/BLOB) must fit in 65,535 bytes, so total VARCHAR width is capped per table, not per column.

PRACTICE — PART FOUR

1. Create a table with a FLOAT and a DECIMAL column, insert 0.1 ten times, sum both, and compare to 1.0. Beginner
2. Design types for a student table: roll number, name, date of birth, CGPA, hostel flag, phone, address. Justify each. Beginner
3. Show that a TIMESTAMP column changes its displayed value when you change `SET time_zone = '+00:00';` while a DATETIME does not. Intermediate
4. Attempt to create a table with twenty `VARCHAR(5000)` columns and explain the error. Advanced

PART FIVE

Create and manage tables

The full lifecycle of a table, from an empty schema to a safely evolved one.

5.1 CREATE DATABASE

```
CREATE DATABASE IF NOT EXISTS shop
  CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
SHOW DATABASES;
SHOW CREATE DATABASE shop;
ALTER DATABASE shop CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_as_cs;
DROP DATABASE IF EXISTS shop;     -- irreversible: every table goes
```

A database's character set and collation are only *defaults* inherited by new tables. Changing them later does not convert existing tables — you must `ALTER TABLE t CONVERT TO CHARACTER SET utf8mb4;` each one.

5.2 CREATE TABLE in full

```
CREATE TABLE IF NOT EXISTS students (
  student_id   INT UNSIGNED AUTO_INCREMENT,
  roll_no      CHAR(10)      NOT NULL,
  full_name    VARCHAR(80)   NOT NULL,
  email        VARCHAR(254)  NOT NULL,
  dob          DATE          NOT NULL,
  cgpa         DECIMAL(4,2)  NOT NULL DEFAULT 0.00,
  branch       ENUM('CSE','ECE','MECH','CIVIL') NOT NULL,
  is_hosteller BOOLEAN       NOT NULL DEFAULT FALSE,
  admitted_on  TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  age          INT AS (TIMESTAMPDIFF(YEAR, dob, CURDATE())) VIRTUAL,
  PRIMARY KEY (student_id),
  UNIQUE  KEY uq_students_roll  (roll_no),
  UNIQUE  KEY uq_students_email (email),
  KEY     idx_students_branch   (branch, cgpa),
  CONSTRAINT chk_students_cgpa CHECK (cgpa BETWEEN 0 AND 10)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='Enrolled students';
Read that statement clause by clause; it contains almost everything Part 5 and
Part 6 teach. Three details worth naming now: UNSIGNED doubles the positive range
and forbids negative ids; the generated column age is computed on read (VIRTUAL)
rather than stored, so it never goes stale; and KEY in a CREATE TABLE means
index, not constraint.
```

Inspecting what you built

```
DESC students;                       -- columns, types, null, key, default
SHOW CREATE TABLE students\G         -- the authoritative definition
SHOW INDEX FROM students;            -- every index and its cardinality
SHOW TABLE STATUS LIKE 'students'\G  -- rows, engine, size, collation
SELECT column_name, data_type, is_nullable, column_default
  FROM information_schema.columns
 WHERE table_schema = 'company' AND table_name = 'students';
```

information_schema is the standard, queryable catalogue — the same metadata as SHOW, but as tables you can join and filter. Analysts use it to audit schemas: find every column named like '%email%', every table without a primary key, every index never used.

5.3 ALTER TABLE — the complete catalogue

GOAL	STATEMENT
Add a column	ALTER TABLE students ADD COLUMN phone VARCHAR(15) NULL;
Add at a position	… ADD COLUMN nickname VARCHAR(30) AFTER full_name;
Add several at once	… ADD COLUMN a INT, ADD COLUMN b DATE;
Change type / nullability	… MODIFY COLUMN phone VARCHAR(20) NOT NULL;
Rename + redefine	… CHANGE COLUMN phone mobile VARCHAR(20) NOT NULL;
Rename only	… RENAME COLUMN mobile TO mobile_no;
Set / drop a default	… ALTER COLUMN cgpa SET DEFAULT 0; … DROP DEFAULT;
Drop a column	… DROP COLUMN nickname;
Add an index	… ADD INDEX idx_students_dob (dob);
Add a unique constraint	… ADD CONSTRAINT uq_phone UNIQUE (mobile_no);
Add a foreign key	… ADD CONSTRAINT fk_s_branch FOREIGN KEY (branch_id) REFERENCES branches(id);
Drop a constraint	… DROP INDEX uq_phone; … DROP FOREIGN KEY fk_s_branch; … DROP CHECK chk_students_cgpa;
Rename the table	ALTER TABLE students RENAME TO learners;
Change engine / charset	… ENGINE=InnoDB; … CONVERT TO CHARACTER SET utf8mb4;
Reset auto-increment	… AUTO_INCREMENT = 5001;
Control the algorithm	… ADD COLUMN x INT, ALGORITHM=INSTANT, LOCK=NONE;

Adding a NOT NULL column to a populated table

This is the classic failure. The existing rows have no value for the new column, so the engine must invent one. Three correct approaches:

```
-- 1. Give it a default
ALTER TABLE employees ADD COLUMN country CHAR(2) NOT NULL DEFAULT 'IN';

-- 2. Three-step backfill (the safe production pattern)
ALTER TABLE employees ADD COLUMN country CHAR(2) NULL;
UPDATE employees SET country = 'IN' WHERE country IS NULL;
ALTER TABLE employees MODIFY COLUMN country CHAR(2) NOT NULL;

-- 3. Derive it, if it is a function of other columns
ALTER TABLE employees
  ADD COLUMN name_upper VARCHAR(60) AS (UPPER(name)) STORED;
```

5.4 DROP and TRUNCATE in practice

```
-- Refuse-safe order: children before parents
DROP TABLE IF EXISTS order_items;
DROP TABLE IF EXISTS orders;

-- Or suspend checks for a bulk reset (development only)
SET FOREIGN_KEY_CHECKS = 0;
TRUNCATE TABLE order_items;
TRUNCATE TABLE orders;
SET FOREIGN_KEY_CHECKS = 1;

-- Rename-then-drop: reversible for a few minutes
RENAME TABLE orders TO orders_deprecated_2026_08;
```

REAL-WORLD EXAMPLE

Professional teams almost never DROP a table in the same release that stops using it. They rename it with a date suffix, deploy, watch for errors for a week, then drop it. The same discipline applies to columns: stop writing, deploy, stop reading, deploy, then drop. A dropped column cannot be un-dropped without a restore.

PRACTICE — PART FIVE

1. Build the `students` table from 5.2 and confirm the generated `age` column updates when you change a DOB. Beginner
2. Add a NOT NULL `section` column to a populated table using the three-step backfill. Intermediate
3. Use `information_schema` to list every table in your database that has no primary key. Advanced

PART SIX

Keys and constraints

Constraints are the promises your database keeps even when every application talking to it has a bug. This part is dense with interview material.

6.1 The key vocabulary, precisely

Consider a table of employees with columns *emp_id*, *email*, *pan_no*, *name*, *dept_id*, and suppose emp_id, email and pan_no are each unique.

KEY	DEFINITION	IN THIS TABLE
Super key	Any set of columns that uniquely identifies a row — including sets with redundant columns	{emp_id}, {emp_id, name}, {email, dept_id}, {emp_id, email, name} …
Candidate key	A *minimal* super key: remove any column and uniqueness is lost	{emp_id}, {email}, {pan_no}
Primary key	The one candidate key chosen to identify rows; implies NOT NULL + UNIQUE	{emp_id}
Alternate key	Every candidate key not chosen as primary	{email}, {pan_no}
Composite key	Any key made of two or more columns	{order_id, product_id} in order_items
Compound key	A composite key whose parts are themselves keys elsewhere	{order_id, product_id} — both are foreign keys
Foreign key	A column whose values must exist in another table's key	{dept_id} → departments.dept_id
Surrogate key	A meaningless generated identifier	emp_id AUTO_INCREMENT
Natural key	A key with business meaning	pan_no, email
Unique key	The implementation of a candidate key as a constraint; allows NULLs in MySQL	uq_email

INTERVIEW TIP

The chain to memorise: *every candidate key is a super key; the primary key is one chosen candidate key; the rest are alternate keys.* If asked "how many super keys does a table with a single-column primary key and n other columns have?", the answer is 2^n — every subset of the other columns combined with the primary key.

6.2 PRIMARY KEY

Definition. The column or set of columns that uniquely identifies each row. Exactly one per table. Implicitly NOT NULL and UNIQUE, and in InnoDB it is also the *clustered index* — the physical order in which rows are stored.

```
-- inline
CREATE TABLE dept (dept_id INT PRIMARY KEY, dept_name VARCHAR(50));
-- named table constraint (preferred: you can drop it by name)
CREATE TABLE dept (
  dept_id INT, dept_name VARCHAR(50),
  CONSTRAINT pk_dept PRIMARY KEY (dept_id)
);
-- composite
CREATE TABLE enrolments (
  student_id INT, course_id INT, grade CHAR(2),
  PRIMARY KEY (student_id, course_id)
);
ALTER TABLE dept DROP PRIMARY KEY;
ALTER TABLE dept ADD PRIMARY KEY (dept_id);
```

Real-world use. Because the primary key is the clustered index, every secondary index stores a copy of it. A wide primary key (a 36-character UUID string, a three-column natural key) is therefore paid for in *every* index on the table. This is the strongest technical argument for narrow surrogate keys.

Common mistakes. Using an email or phone number as the primary key — people change both, and updating a primary key cascades pain through every referencing table. Using a business identifier that turns out to be reissued (roll numbers, employee codes after rehire).

Interview question. *Can a primary key contain NULL?* No, never — not even one column of a composite one. Beginner

6.3 UNIQUE

ASPECT	PRIMARY KEY	UNIQUE
Per table	One	Many
NULLs	Not allowed	Allowed — and multiple NULLs are allowed in MySQL, PostgreSQL and Oracle
Index type	Clustered in InnoDB	Secondary, non-clustered
Purpose	Row identity	Business-rule uniqueness
Referenceable by FK	Yes	Yes

```
CREATE TABLE users (
  user_id  INT AUTO_INCREMENT PRIMARY KEY,
  email    VARCHAR(254) NOT NULL UNIQUE,
  phone    VARCHAR(15)  NULL UNIQUE,        -- many rows may have NULL phone
  UNIQUE KEY uq_user_org (org_id, username) -- composite: unique per org
);
INSERT INTO users (email, phone) VALUES ('a@x.com', NULL), ('b@x.com', NULL);
-- both succeed: NULL is not equal to NULL, so no duplicate is detected
```

That last behaviour surprises everyone once. If you need "at most one row without a phone", a unique constraint cannot express it; use a generated column or an application rule. Note also that a composite unique key constrains the *combination*: `(1,'ravi')` and `(2,'ravi')` both fit `uq_user_org`.

Dialect note. SQL Server allows only *one* NULL per unique constraint — a genuine portability trap. A filtered index (`WHERE phone IS NOT NULL`) is the SQL Server workaround.

6.4 NOT NULL, DEFAULT and CHECK

```
CREATE TABLE bookings (
  booking_id  INT AUTO_INCREMENT PRIMARY KEY,
  guest_name  VARCHAR(80)  NOT NULL,
  guests      TINYINT      NOT NULL DEFAULT 1,
  status      VARCHAR(10)  NOT NULL DEFAULT 'NEW',
  check_in    DATE         NOT NULL,
  check_out   DATE         NOT NULL,
  price       DECIMAL(10,2) NOT NULL,
  discount    DECIMAL(10,2) NOT NULL DEFAULT 0,
```

```
  created_at  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT chk_dates    CHECK (check_out > check_in),
  CONSTRAINT chk_guests   CHECK (guests BETWEEN 1 AND 8),
  CONSTRAINT chk_discount CHECK (discount <= price),
  CONSTRAINT chk_status   CHECK (status IN ('NEW','PAID','CANCELLED'))
);
```

CONSTRAINT	GUARANTEES	NOTES AND TRAPS
NOT NULL	A value is always present	Default your columns NOT NULL and justify each exception; NULL is the source of most query bugs (Part 9)
DEFAULT	A value when the INSERT omits the column	Applies only on omission — explicitly inserting NULL stores NULL (or errors if NOT NULL). Expression defaults need parentheses: DEFAULT (UUID())
CHECK	A row-level boolean rule	Enforced from MySQL 8.0.16 (parsed and *ignored* before that); cannot reference other tables or use subqueries or non-deterministic functions

COMMON MISTAKE

A CHECK constraint is satisfied when it evaluates to TRUE *or to UNKNOWN*. CHECK (guests > 0) does **not** reject a NULL guests, because NULL > 0 is UNKNOWN, not FALSE. Pair every CHECK with NOT NULL when you mean it.

6.5 FOREIGN KEY and referential integrity

Referential integrity is the guarantee that every reference points at something real: no order belongs to customer 999 if customer 999 does not exist, and no customer can be deleted while their orders survive. A foreign key is how you declare it. The referenced column must be a primary key or a unique key, the types must match, and in InnoDB both sides need an index (the child's is created automatically if absent).

```
CREATE TABLE orders (
  order_id INT AUTO_INCREMENT PRIMARY KEY,
  cust_id  INT NOT NULL,
  amount   DECIMAL(10,2) NOT NULL,
  CONSTRAINT fk_orders_cust
    FOREIGN KEY (cust_id) REFERENCES customers(cust_id)
    ON DELETE RESTRICT
```

```
    ON UPDATE CASCADE
);

INSERT INTO orders (cust_id, amount) VALUES (999, 500);
-- ERROR 1452: Cannot add or update a child row: a foreign key constraint
fails
DELETE FROM customers WHERE cust_id = 1;
-- ERROR 1451: Cannot delete or update a parent row
```

Referential actions

ACTION	ON PARENT DELETE	ON PARENT UPDATE OF THE KEY	USE WHEN
RESTRICT / NO ACTION	Refuse if children exist	Refuse	Default and safest — forces the application to decide
CASCADE	Delete the children too	Update the children's values	Children have no life of their own: order_items with their order
SET NULL	Set the child's FK to NULL	Set to NULL	The link is optional: an employee keeps existing when their department closes. The column must be nullable
SET DEFAULT	Set to the column default	Same	Parsed but not implemented by InnoDB — do not rely on it in MySQL

```
-- Lines die with their order; the order keeps its customer
ALTER TABLE order_items ADD CONSTRAINT fk_oi_order
  FOREIGN KEY (order_id) REFERENCES orders(order_id)
  ON DELETE CASCADE ON UPDATE CASCADE;

-- Employees survive a closed department
ALTER TABLE employees ADD CONSTRAINT fk_emp_dept
  FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
  ON DELETE SET NULL ON UPDATE CASCADE;

-- Inspect what exists
SELECT constraint_name, table_name, referenced_table_name, delete_rule,
update_rule
  FROM information_schema.referential_constraints
 WHERE constraint_schema = 'company';
```

IMPORTANT

ON DELETE CASCADE is powerful and dangerous. Cascades chain: deleting one customer can silently remove their orders, those orders' items, and the payment rows referencing them. Triggers do *not* fire for cascaded deletes in MySQL, so audit trails miss them. Use CASCADE only for true ownership relationships, and never on a table you might one day need to reconstruct.

Common mistakes. Mismatched types (INT child, INT UNSIGNED parent) causing error 3780; creating a foreign key on a MyISAM table where it is silently ignored; loading data in the wrong order without disabling checks; and self-referencing keys with NOT NULL, which make the first insert impossible.

Interview question. *Can a foreign key reference a non-unique column?* No — the target must be a primary key or unique key, otherwise "the parent row" would be ambiguous. Intermediate

6.6 A complete constrained schema

Everything in this part, applied to a small library system. Notice that the rules a beginner would put in application code are all in the schema here.

```
CREATE TABLE members (
  member_id  INT AUTO_INCREMENT PRIMARY KEY,
  email      VARCHAR(254) NOT NULL,
  full_name  VARCHAR(80)  NOT NULL,
  joined_on  DATE         NOT NULL DEFAULT (CURRENT_DATE),
  is_active  BOOLEAN      NOT NULL DEFAULT TRUE,
  CONSTRAINT uq_members_email UNIQUE (email),
  CONSTRAINT chk_members_email CHECK (email LIKE '%_@_%._%')
);

CREATE TABLE books (
  book_id   INT AUTO_INCREMENT PRIMARY KEY,
  isbn      CHAR(13)     NOT NULL,
  title     VARCHAR(200) NOT NULL,
  copies    SMALLINT     NOT NULL DEFAULT 1,
  CONSTRAINT uq_books_isbn UNIQUE (isbn),
  CONSTRAINT chk_books_copies CHECK (copies >= 0)
);

CREATE TABLE loans (
  loan_id    INT AUTO_INCREMENT PRIMARY KEY,
  book_id    INT  NOT NULL,
  member_id  INT  NOT NULL,
  issued_on  DATE NOT NULL DEFAULT (CURRENT_DATE),
  due_on     DATE NOT NULL,
  returned_on DATE NULL,
  CONSTRAINT fk_loans_book   FOREIGN KEY (book_id)   REFERENCES books(book_id)
```

```
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_loans_member FOREIGN KEY (member_id) REFERENCES
members(member_id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT chk_loans_due   CHECK (due_on > issued_on),
  CONSTRAINT chk_loans_ret   CHECK (returned_on IS NULL OR returned_on >=
issued_on),
  KEY idx_loans_member_open (member_id, returned_on)
);
```

Two rules this schema still cannot express: "a member may hold at most three open loans" and "copies on loan may not exceed copies owned". Neither is a row-level condition, so neither is a CHECK. They need a trigger (Part 29) or a transaction with an explicit lock (Part 26) — a very good interview discussion.

CHAPTER SUMMARY

- Super key ² candidate key ² primary key; the unchosen candidates are alternate keys.
- PRIMARY KEY = NOT NULL + UNIQUE + clustered index; keep it narrow and meaningless.
- UNIQUE permits multiple NULLs in MySQL (one in SQL Server) and constrains combinations when composite.
- CHECK passes on UNKNOWN, so NULL slips through unless the column is NOT NULL.
- Foreign keys give referential integrity; RESTRICT is the safe default, CASCADE is for owned children, SET NULL for optional links.
- Declare rules in the schema whenever they are row-level; use triggers or transactions for set-level rules.

INTERVIEW QUESTIONS — PART SIX

1. **PRIMARY KEY vs UNIQUE?** Beginner One per table vs many; no NULLs vs NULLs allowed; clustered vs secondary index.
2. **Difference between a candidate key and a super key?** Beginner A candidate key is a minimal super key.
3. **Can a table have two primary keys?** Beginner No. It can have one composite primary key over two columns — a common trick question.
4. **What is referential integrity?** Intermediate Every foreign key value either matches an existing parent key or is NULL.
5. **Difference between ON DELETE CASCADE and SET NULL?** Intermediate CASCADE deletes the children; SET NULL keeps them and nulls the reference, requiring a nullable column.
6. **Why is a UUID a poor clustered primary key?** Advanced Random values scatter inserts across the B-tree, causing page splits and poor cache locality, and every secondary index carries the wide key.
7. **Do triggers fire on cascaded deletes in MySQL?** Advanced No — a real auditing pitfall.
8. **How do you enforce "at most three open loans per member"?** Advanced Not with CHECK; use a BEFORE INSERT trigger counting open loans, or a transaction with `SELECT … FOR UPDATE`.

PRACTICE — PART SIX

1. Build the library schema from 6.6 and try to break each constraint deliberately; record every error number. Beginner
2. List all candidate keys of a table with columns (emp_id, email, pan, aadhaar, name). Beginner
3. Create two versions of a parent/child pair, one with CASCADE and one with SET NULL, and delete a parent from each. Intermediate
4. Show that `CHECK (qty > 0)` accepts a NULL qty, then fix it. Intermediate
5. Add a self-referencing manager foreign key to a table that already has rows pointing at non-existent managers. Explain the failure and fix the data first. Advanced

PART SEVEN

Insert, update and delete

Writing data is easier than reading it and far more dangerous. Every statement in this part can destroy a table if you mistype one clause.

7.1 INSERT — every form

```
-- 1. Named columns (always do this)
INSERT INTO departments (dept_id, dept_name, location)
VALUES (60, 'Legal', 'Delhi');

-- 2. Positional: fragile, breaks the day someone adds a column
INSERT INTO departments VALUES (70, 'Design', 'Goa');

-- 3. Multiple rows in one statement: one round trip, one transaction
INSERT INTO departments (dept_id, dept_name, location) VALUES
  (80, 'Support',  'Chennai'),
  (90, 'Logistics','Nagpur'),
  (100,'Training', NULL);

-- 4. INSERT ... SELECT: copy or derive rows from a query
INSERT INTO high_earners (emp_id, name, salary)
SELECT emp_id, name, salary FROM employees WHERE salary > 100000;

-- 5. INSERT ... SET  (MySQL only, readable for wide tables)
INSERT INTO departments SET dept_id = 110, dept_name = 'Audit';

-- 6. Default row
INSERT INTO audit_log () VALUES ();

-- 7. Upsert: insert, or update if the key exists
INSERT INTO products (product_id, product_name, price)
VALUES (1, 'Mechanical Keyboard', 4800)
ON DUPLICATE KEY UPDATE price = VALUES(price);   -- MySQL 8: price =
new.price

-- 8. Ignore duplicates instead of failing (careful: hides other errors too)
INSERT IGNORE INTO departments (dept_id, dept_name) VALUES (10,
'Engineering');

-- 9. Replace: DELETE the old row then INSERT. Fires delete triggers,
--    breaks cascading children, resets omitted columns to defaults.
```

```
REPLACE INTO products (product_id, product_name, price) VALUES
(1,'Keyboard',4800);
```

FORM	BEHAVIOUR ON AN EXISTING KEY	RECOMMENDED?
INSERT	Error 1062	Yes — the default, and errors are informative
INSERT IGNORE	Skips the row silently; warnings only	Only for bulk loads where duplicates are expected
ON DUPLICATE KEY UPDATE	Updates the named columns	Yes — the correct upsert
REPLACE	Deletes then inserts; loses unnamed columns	Rarely — almost always a bug in disguise

AUTO_INCREMENT and the id you just created

```
INSERT INTO customers (name, city, signup_date)
VALUES ('Neha Kulkarni', 'Nashik', CURDATE());
SELECT LAST_INSERT_ID();      -- the id of the row this session just inserted

INSERT INTO orders (cust_id, order_date, amount)
VALUES (LAST_INSERT_ID(), CURDATE(), 4999);   -- link child to fresh parent
```

LAST_INSERT_ID() is per-connection, so concurrent users cannot see each other's value. For a multi-row insert it returns the id of the *first* row. Auto-increment values are never reused after a rollback or a deletion — gaps in the sequence are normal and not a bug.

Dialect note. PostgreSQL upserts with INSERT … ON CONFLICT (col) DO UPDATE SET … and returns generated values with RETURNING id. Oracle and SQL Server use MERGE; SQL Server has SCOPE_IDENTITY() and an OUTPUT clause.

7.2 UPDATE

```
-- Single column, single row
UPDATE employees SET salary = 99000 WHERE emp_id = 2;

-- Several columns at once; the SET list is evaluated left to right in MySQL
UPDATE employees
   SET salary = salary * 1.10,
       city   = 'Bengaluru'
 WHERE dept_id = 10 AND hire_date < '2020-01-01';
```

```
-- Conditional per row with CASE
UPDATE employees
   SET salary = CASE
         WHEN salary < 70000  THEN salary * 1.15
         WHEN salary < 100000 THEN salary * 1.10
         ELSE salary * 1.05
       END;

-- Update from another table (join form)
UPDATE employees e
  JOIN departments d ON e.dept_id = d.dept_id
   SET e.city = d.location
 WHERE e.city IS NULL;

-- Update from an aggregate (correlated subquery)
UPDATE departments d
   SET d.headcount = (SELECT COUNT(*) FROM employees e WHERE e.dept_id =
d.dept_id);

-- Bounded and ordered update
UPDATE employees SET grade = 'A' ORDER BY salary DESC LIMIT 3;
```

The row count MySQL reports distinguishes *matched* from *changed*: Rows matched: 5 Changed: 3 means two rows already held the new value. If your update "did nothing", read that line before doubting the WHERE clause.

IMPORTANT — THE FOUR-STEP SAFE UPDATE

1. Write it as a SELECT with the identical WHERE and read the rows.
2. START TRANSACTION;
3. Run the UPDATE and check the reported row count against step 1.
4. COMMIT; — or ROLLBACK; if the count surprised you.

7.3 DELETE

```
DELETE FROM orders WHERE order_id = 4;                 -- one row
DELETE FROM orders WHERE status = 'CANCELLED';          -- a set
DELETE FROM orders WHERE order_date < '2022-01-01' LIMIT 1000;   -- batched

-- Delete with a join: remove order items of cancelled orders
DELETE oi
  FROM order_items oi
  JOIN orders o ON o.order_id = oi.order_id
 WHERE o.status = 'CANCELLED';
```

```
-- Anti-join delete: customers who never ordered
DELETE c FROM customers c
 LEFT JOIN orders o ON o.cust_id = c.cust_id
 WHERE o.order_id IS NULL;

-- Delete duplicates, keeping the lowest id (MySQL 8 needs the join form)
DELETE e1 FROM employees e1
  JOIN employees e2
    ON e1.email = e2.email AND e1.emp_id > e2.emp_id;
```

Deleting millions of rows in one statement builds an enormous undo log, holds locks for minutes and can stall replication. The professional pattern is a batched loop — DELETE … LIMIT 5000 repeated until zero rows are affected, committing between batches.

COMMON MISTAKE — THE TWO STATEMENTS THAT END CAREERS

```
UPDATE employees SET salary = 50000;   -- every employee
DELETE FROM employees;                 -- every row
```

Both are valid SQL and both execute instantly. Defences, in order of reliability: a restorable backup, Safe Updates in your client, explicit transactions, a read-only account for exploration, and the habit of typing the WHERE clause *before* the SET clause.

7.4 Soft deletes

Most production systems do not delete anything. They mark rows inactive, so history, audits and foreign keys survive:

```
ALTER TABLE customers ADD COLUMN deleted_at DATETIME NULL;
UPDATE customers SET deleted_at = NOW() WHERE cust_id = 3;   -- "delete"
SELECT * FROM customers WHERE deleted_at IS NULL;            -- live rows
CREATE VIEW active_customers AS
  SELECT * FROM customers WHERE deleted_at IS NULL;          -- hide the
filter
```

The cost is that every query must remember the filter — which is why the view exists. Weigh it against regulatory deletion requirements: a soft delete does not satisfy a legal "erase my data" request.

INTERVIEW QUESTIONS — PART SEVEN

1. **How do you insert multiple rows in one statement?** Beginner One VALUES clause with comma-separated tuples — also far faster than N statements.
2. **Difference between REPLACE and ON DUPLICATE KEY UPDATE?** Intermediate REPLACE deletes the old row (firing delete triggers, cascading to children, resetting omitted columns); ODKU updates in place.
3. **What does LAST_INSERT_ID() return after a 10-row insert?** Advanced The id of the first row inserted.
4. **Why does my UPDATE report "Rows matched: 5, Changed: 0"?** Intermediate The rows already contained those values; MySQL skips no-op writes.
5. **How do you delete duplicate rows keeping one?** Advanced Self-join on the duplicate key with an id inequality, or a window function `ROW_NUMBER()` in a CTE (Part 18).
6. **Is DELETE without WHERE the same as TRUNCATE?** Beginner Same visible outcome, entirely different mechanism: DELETE is transactional, logged per row, fires triggers and preserves auto-increment.

PRACTICE — PART SEVEN

1. Insert three new departments in one statement and one product with an upsert that changes the price of an existing product. Beginner
2. Give every employee in Sales a 12% raise inside a transaction, verify the count, then roll back. Beginner
3. Copy all delivered orders older than 2024 into an `orders_archive` table with INSERT … SELECT, then delete them in batches of 100. Intermediate
4. Set each employee's city to their department's location where the city is NULL, using the join form of UPDATE. Intermediate
5. Add a duplicate email to employees (drop the unique key first), then remove the duplicate with a self-join delete. Advanced

PART EIGHT

SELECT from zero

One statement, nine clauses, and the order in which the database actually runs them.

8.1 The anatomy of a SELECT

```
SELECT   DISTINCT  d.dept_name, COUNT(*) AS headcount      -- 6, 7
FROM     employees e                                      -- 1
JOIN     departments d ON d.dept_id = e.dept_id           -- 2
WHERE    e.hire_date >= '2018-01-01'                      -- 3
GROUP BY d.dept_name                                      -- 4
HAVING   COUNT(*) > 1                                     -- 5
ORDER BY headcount DESC, d.dept_name                      -- 8
LIMIT    10 OFFSET 0;                                     -- 9
```

#	CLAUSE	WHAT IT DOES	CAN IT SEE SELECT ALIASES?
1	FROM	Names the source tables and produces the starting row set	—
2	JOIN … ON	Combines rows from multiple tables	No
3	WHERE	Filters individual rows	No
4	GROUP BY	Collapses rows into groups	MySQL only, as an extension
5	HAVING	Filters groups	MySQL yes; standard SQL no
6	SELECT	Computes the output columns	Defines them
7	DISTINCT	Removes duplicate output rows	Yes
8	ORDER BY	Sorts the result	Yes
9	LIMIT / OFFSET	Takes a slice	—

This is the *logical* order, and it is the single most useful thing to know about SQL. It explains why a WHERE clause cannot reference an alias, why HAVING can use aggregates but WHERE cannot, why ORDER BY can use an alias, and why LIMIT 10

after an ORDER BY gives the top ten rather than ten arbitrary rows. The optimizer may execute things in any physical order it likes, as long as the answer matches this model.

8.2 SELECT * and why not to ship it

```
SELECT * FROM employees;                    -- every column, every row
SELECT e.* FROM employees e JOIN departments d USING (dept_id);
SELECT name, salary, hire_date FROM employees;   -- what production code does
```

SELECT * is excellent while exploring and a liability in code. It transfers columns nobody uses (including large TEXT and BLOB values), prevents *covering indexes* from satisfying the query from the index alone (Part 23), and silently changes the shape of your result when someone alters the table — breaking positional code and ORM mappings. Name your columns.

8.3 DISTINCT

```
SELECT DISTINCT city FROM employees;              -- unique cities
SELECT DISTINCT dept_id, city FROM employees;     -- unique COMBINATIONS
SELECT COUNT(DISTINCT dept_id) FROM employees;    -- how many distinct depts
SELECT DISTINCT COUNT(*) FROM employees;          -- pointless: one row
anyway
```

```
+-----------+        +---------+-----------+
| city      |        | dept_id | city      |
+-----------+        +---------+-----------+
| Bengaluru |        |      10 | Bengaluru |
| Hyderabad |        |      10 | Hyderabad |
| Mumbai    |        |      20 | Mumbai    |
| Ahmedabad |        |      20 | Ahmedabad |
| Pune      |        |      30 | Pune      |
| Delhi     |        |      40 | Bengaluru |
+-----------+        |    NULL | Delhi     |
6 rows               +---------+-----------+
```

DISTINCT applies to the **whole select list**, never to one column — there is no such thing as SELECT DISTINCT(a), b meaning "distinct a"; the parentheses are decoration. Note also that DISTINCT treats all NULLs as equal to each other, so they collapse into one row, even though NULL = NULL is not true (Part 9).

COMMON MISTAKE

Adding DISTINCT to "fix" duplicate rows after a join. The duplicates are telling you something — usually that you joined a one-to-many relationship and multiplied rows. DISTINCT hides the symptom, costs a sort or hash of the entire result, and will give wrong sums the moment you aggregate. Find the join that fanned out (Part 14) instead.

8.4 Aliases

```
-- Column aliases: AS is optional but always write it
SELECT name AS employee_name,
       salary * 12 AS annual_ctc,
       salary / 100000 AS "salary in lakhs"     -- quotes needed for spaces
  FROM employees;

-- Table aliases: mandatory for self-joins, invaluable everywhere
SELECT e.name AS employee, m.name AS manager
  FROM employees e
  LEFT JOIN employees m ON m.emp_id = e.manager_id;

-- Alias visibility
SELECT salary * 12 AS ctc FROM employees WHERE ctc > 1000000;  -- ERROR 1054
SELECT salary * 12 AS ctc FROM employees ORDER BY ctc DESC;    -- works
SELECT salary * 12 AS ctc FROM employees HAVING ctc > 1000000; -- works in
MySQL
```

To filter on a computed value portably, either repeat the expression in WHERE or wrap the query: SELECT * FROM (SELECT salary*12 AS ctc FROM employees) t WHERE ctc > 1000000;. Once a table alias is declared, the original name is no longer usable in that query — a frequent source of error 1054.

8.5 Calculated columns and expressions

```
SELECT name,
       salary,
       salary * 12                              AS annual,
       ROUND(salary * 0.1, 2)                   AS bonus,
       salary + ROUND(salary * 0.1, 2)          AS total,
       CONCAT(name, ' (', IFNULL(city,'n/a'), ')') AS label,
       TIMESTAMPDIFF(YEAR, hire_date, CURDATE()) AS years_served,
       CASE WHEN salary > 100000 THEN 'Senior' ELSE 'Standard' END AS band,
       CURDATE()                                AS report_date,
       'Company Ltd'                            AS company     -- literal
column
```

```
  FROM employees
 WHERE dept_id = 10;
```

A SELECT list may contain any expression: column references, literals, arithmetic, function calls, CASE expressions, even scalar subqueries. Nothing is stored; the values are computed per row at query time. If an expression is expensive and queried constantly, promote it to a STORED generated column and index it.

A query needs no table at all — useful for testing functions: SELECT NOW(), 5*3, UPPER('sql');. In Oracle you must write SELECT SYSDATE FROM dual;.

INTERVIEW QUESTIONS — PART EIGHT

1. **Give the logical order of SELECT clause evaluation.** Intermediate FROM, JOIN, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, LIMIT.
2. **Why is SELECT * discouraged?** Beginner Unnecessary I/O, defeats covering indexes, and breaks callers when the schema changes.
3. **Does DISTINCT apply per column?** Beginner No — to the entire select list.
4. **How do you filter on a computed alias portably?** Intermediate Repeat the expression, or wrap the query in a derived table.
5. **Is DISTINCT or GROUP BY faster for unique values?** Advanced Usually identical — MySQL's optimizer treats them the same — but GROUP BY may use an index for ordering, and DISTINCT expresses intent more clearly when no aggregate is involved.

PART NINE

Filtering data

Every predicate you will ever write, and the three-valued logic that trips up even experienced developers.

9.1 WHERE

```
SELECT name, salary FROM employees WHERE salary > 90000;
SELECT * FROM employees WHERE city = 'Mumbai';
SELECT * FROM employees WHERE hire_date >= '2020-01-01';
SELECT * FROM orders   WHERE status <> 'CANCELLED';     -- or != in MySQL
```

WHERE evaluates its condition once per candidate row and keeps the row only when the result is **TRUE**. Not "not false" — TRUE. That distinction is the whole of section 9.7.

9.2 AND, OR, NOT

```
SELECT * FROM employees WHERE dept_id = 10 AND salary > 90000;
SELECT * FROM employees WHERE dept_id = 10 OR  dept_id = 20;
SELECT * FROM employees WHERE NOT dept_id = 10;          -- excludes NULL
dept!
SELECT * FROM employees
 WHERE (dept_id = 10 OR dept_id = 20) AND salary > 80000;   -- parenthesise!
```

AND binds tighter than OR, so a OR b AND c means a OR (b AND c). Relying on precedence is a bug waiting to be introduced by the next person who edits the query; parenthesise every mixed condition, always.

A	B	A AND B	A OR B	NOT A
TRUE	TRUE	TRUE	TRUE	FALSE
TRUE	FALSE	FALSE	TRUE	FALSE
TRUE	NULL	NULL	TRUE	FALSE
FALSE	FALSE	FALSE	FALSE	TRUE
FALSE	NULL	FALSE	NULL	TRUE
NULL	NULL	NULL	NULL	NULL

Read the two most surprising rows carefully: FALSE AND NULL is FALSE (nothing can rescue a false), and TRUE OR NULL is TRUE (nothing can spoil a true). Everything else involving NULL is NULL.

9.3 IN and NOT IN

```
SELECT * FROM employees WHERE dept_id IN (10, 20, 30);
SELECT * FROM employees WHERE city NOT IN ('Delhi', 'Kochi');
SELECT * FROM employees                          -- IN with a subquery
 WHERE dept_id IN (SELECT dept_id FROM departments WHERE
location='Bengaluru');
SELECT * FROM order_items                        -- row constructor
 WHERE (order_id, product_id) IN ((1,1), (6,4));
```

x IN (a,b,c) is shorthand for x=a OR x=b OR x=c, and that expansion explains the notorious trap:

COMMON MISTAKE — NOT IN WITH NULL

```
SELECT * FROM departments
 WHERE dept_id NOT IN (SELECT dept_id FROM employees);   -- returns NOTHING
```

Omar has a NULL dept_id, so the list is (10,10,10,20,20,20,30,30,40,NULL). 50 NOT IN (…,NULL) expands to 50<>10 AND … AND 50<>NULL, whose last term is UNKNOWN, so the whole conjunction is UNKNOWN and the row is dropped. Fix it with WHERE dept_id IS NOT NULL inside the subquery, or use NOT EXISTS, which is NULL-safe.

9.4 BETWEEN and NOT BETWEEN

```
SELECT * FROM employees WHERE salary BETWEEN 70000 AND 100000;  -- inclusive
SELECT * FROM employees WHERE salary NOT BETWEEN 70000 AND 100000;
SELECT * FROM orders WHERE order_date BETWEEN '2023-01-01' AND '2023-12-31';
```

BETWEEN is inclusive on both ends and equivalent to x >= low AND x <= high. The bounds must be in that order: BETWEEN 100000 AND 70000 returns nothing at all rather than an error.

COMMON MISTAKE — BETWEEN ON DATETIMES

created_at BETWEEN '2023-01-01' AND '2023-01-31' silently loses 31 January, because the bare date means midnight and every event later that day exceeds it. Use a half-open range instead: created_at >= '2023-01-01' AND created_at < '2023-02-01'. This form is also index-friendly, unlike DATE(created_at) = …, which wraps the column in a function and disables the index.

9.5 LIKE and wildcards

PATTERN	MATCHES	INDEX USABLE?
'A%'	Starts with A	Yes — a range scan
'%a'	Ends with a	No
'%ee%'	Contains ee anywhere	No
'_riya'	Any one character then 'riya'	No
'A____'	A followed by exactly four characters	Yes (prefix)
'50\%%'	Starts with the literal text 50%	Yes

```
SELECT name FROM employees WHERE name LIKE 'A%';       -- Arjun
SELECT name FROM employees WHERE name LIKE '%Nair';     -- Priya Nair
SELECT name FROM employees WHERE name LIKE '%ee%';      -- Meera
SELECT name FROM employees WHERE email NOT LIKE '%@corp.in';
SELECT name FROM employees WHERE name LIKE 'M___a%';    -- M + 3 chars + a
SELECT '100%' LIKE '100\%'            AS escaped;       -- 1
SELECT name FROM employees WHERE name REGEXP '^[AP]';   -- regex alternative
```

% matches any number of characters including none; _ matches exactly one. Case sensitivity follows the column's collation, so on the default utf8mb4_0900_ai_ci, LIKE 'a%' also matches 'Arjun'. For genuine free-text search over long columns, LIKE '%term%' does not scale — use a FULLTEXT index and MATCH … AGAINST (Part 23).

Dialect note. PostgreSQL's LIKE is case-sensitive; use ILIKE for insensitive matching. Oracle and SQL Server are governed by their collations too, and SQL Server LIKE supports character classes such as '[A-C]%'.

9.6 NULL: the whole story

NULL is not zero, not an empty string, and not false. It means **no value is recorded** — unknown or not applicable. Because it is unknown, every comparison with it yields UNKNOWN, a third truth value alongside TRUE and FALSE. This is *three-valued logic*, and it has consequences everywhere.

```
SELECT NULL = NULL      AS eq,      -- NULL  (not 1!)
       NULL <> NULL     AS neq,     -- NULL
       NULL = 0         AS zero,    -- NULL
       NULL = ''        AS empty,   -- NULL
       NULL <=> NULL    AS null_safe,   -- 1   MySQL's null-safe equality
       1 + NULL         AS arith,   -- NULL: arithmetic propagates
```

```
       CONCAT('a',NULL) AS cat,     -- NULL: so does CONCAT
       CONCAT_WS('-','a',NULL,'b') AS cat_ws;  -- 'a-b': this one skips NULLs
```

```
-- Testing for NULL: only IS NULL / IS NOT NULL work
SELECT name FROM employees WHERE dept_id IS NULL;        -- Omar
SELECT name FROM employees WHERE dept_id = NULL;         -- 0 rows, no error
SELECT name FROM employees WHERE email IS NOT NULL;

-- Replacing NULL
SELECT name, IFNULL(city, 'Unknown')        AS city1,    -- MySQL
       COALESCE(city, email, 'Unknown')     AS city2,    -- standard, n args
       IF(city IS NULL, 'Unknown', city)    AS city3
  FROM employees;
```

CONTEXT	HOW NULL BEHAVES
WHERE / JOIN ON	Row is dropped — only TRUE passes
CHECK constraint	Row is *accepted* — UNKNOWN counts as satisfied
Arithmetic and most functions	Result is NULL
COUNT(column)	NULLs are not counted; COUNT(*) counts the row
SUM, AVG, MIN, MAX	NULLs are skipped — AVG divides by the non-null count
GROUP BY	All NULLs form one group together
DISTINCT	All NULLs collapse to one row
ORDER BY	MySQL sorts NULLs first ascending; PostgreSQL and Oracle sort them last
UNIQUE index	Multiple NULLs allowed (MySQL, PostgreSQL, Oracle); one only in SQL Server
UNION / INTERSECT	NULLs are treated as equal for duplicate removal

INTERVIEW TIP

Note the inconsistency worth quoting in an interview: GROUP BY, DISTINCT and UNION treat NULLs as *equal to each other*, while = says they are not comparable. Both behaviours are in the standard. AVG's treatment matters commercially: AVG(discount) over 10 rows where 6 are NULL divides by 4, not 10. If a missing discount means zero, write AVG(COALESCE(discount,0)).

INTERVIEW QUESTIONS — PART NINE

1. **Why does `WHERE col = NULL` return nothing?** Beginner The comparison is UNKNOWN, and WHERE keeps only TRUE. Use IS NULL.
2. **Is NULL the same as 0 or ''?** Beginner No — both are values; NULL is the absence of one.
3. **Explain why NOT IN with a NULL in the list returns no rows.** Advanced The expansion contains `x <> NULL`, which is UNKNOWN, making the whole AND chain UNKNOWN.
4. **Difference between = and <=>?** Intermediate `<=>` is MySQL's null-safe equality: `NULL <=> NULL` is 1. Standard SQL spells it `IS NOT DISTINCT FROM`.
5. **COUNT(*) vs COUNT(col)?** Intermediate COUNT(*) counts rows; COUNT(col) counts non-NULL values of that column.
6. **Why is `WHERE DATE(created_at) = '2023-05-01'` slow?** Advanced Wrapping the column in a function makes the predicate non-sargable, so the index cannot be used; use a half-open range.

PRACTICE — PART NINE

1. List employees hired in 2019 or later, earning between 60,000 and 95,000, excluding Mumbai. Beginner
2. Find every employee whose name contains a double letter. Beginner
3. Find departments with no employees, first with NOT IN (observe the failure), then correctly. Intermediate
4. Compute the average discount two ways — with and without COALESCE — and explain the difference. Intermediate
5. Write a predicate matching employees whose email is missing *or* not on the corp.in domain, and prove it catches Omar. Advanced

PART TEN

Sorting and limiting

Ordering, paging, and why page 900 of a listing is slow in every application you have ever used.

10.1 ORDER BY

```
SELECT name, salary FROM employees ORDER BY salary;            -- ASC default
SELECT name, salary FROM employees ORDER BY salary DESC;
SELECT name, dept_id, salary FROM employees
 ORDER BY dept_id ASC, salary DESC;                            -- tie-break
SELECT name, salary * 12 AS ctc FROM employees ORDER BY ctc DESC;  -- alias
ok
SELECT name, salary FROM employees ORDER BY 2 DESC;             -- by
position
SELECT name FROM employees ORDER BY RAND() LIMIT 1;             -- random row
SELECT * FROM tickets
 ORDER BY FIELD(priority,'URGENT','HIGH','MEDIUM','LOW');       -- custom
order
SELECT * FROM employees ORDER BY city IS NULL, city;            -- NULLs last
```

Without ORDER BY a result has **no guaranteed order**. It may look sorted by primary key because that is how InnoDB happened to read it, and then change when an index is added or the plan changes. Never rely on incidental order.

Sorting by column position (ORDER BY 2) is legal and terrible: adding a column to the select list silently changes the sort. The city IS NULL, city idiom works because the boolean sorts 0 before 1; PostgreSQL and Oracle have the cleaner NULLS LAST, which MySQL lacks.

10.2 LIMIT and OFFSET

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 3;          -- top 3
SELECT * FROM employees ORDER BY salary DESC LIMIT 5 OFFSET 5;  -- rows 6-10
SELECT * FROM employees ORDER BY salary DESC LIMIT 5, 5;        -- MySQL
shorthand
SELECT * FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;  -- 2nd
highest
```

IMPORTANT

In the MySQL shorthand LIMIT a, b the *first* number is the offset and the second is the count — the reverse of the keyword form. Mixing them up is a classic off-by-page bug. Prefer the explicit LIMIT b OFFSET a, which is also standard SQL.

Dialect note. Standard SQL and PostgreSQL/Oracle 12c+ write OFFSET 5 ROWS FETCH NEXT 5 ROWS ONLY; SQL Server uses TOP (5) or the same OFFSET/FETCH; older Oracle used ROWNUM.

10.3 Offset pagination and its cost

```
-- page number p, page size s
SELECT * FROM orders ORDER BY order_id LIMIT s OFFSET (p-1)*s;
-- page 1:   LIMIT 20 OFFSET 0
-- page 900: LIMIT 20 OFFSET 17980   <- reads and discards 17,980 rows
```

OFFSET is not a seek; it is "read these rows and throw them away". Cost grows linearly with the page number, which is why deep pages crawl. Worse, if rows are inserted or deleted while a user pages, the window shifts and rows are seen twice or skipped.

10.4 Keyset pagination (seek method)

```
-- First page
SELECT order_id, order_date, amount
  FROM orders
 ORDER BY order_id
 LIMIT 20;

-- Next page: remember the last order_id you showed (say 4021)
SELECT order_id, order_date, amount
  FROM orders
 WHERE order_id > 4021        -- a seek into the index, not a scan
 ORDER BY order_id
 LIMIT 20;

-- Sorting by a non-unique column needs a tie-breaker in the key
SELECT * FROM orders
 WHERE (order_date, order_id) > ('2023-08-21', 5)
 ORDER BY order_date, order_id
 LIMIT 20;
```

Keyset pagination only walks forward through the index at the size of the page, regardless of how deep you go — page 900 costs the same as page 1. Its price: no "jump to page 47" (you can only go next/previous), and the sort column(s) must be indexed and effectively unique when combined.

	OFFSET (LIMIT/OFFSET)	KEYSET (SEEK)
Deep page cost	Grows with page number	Constant
Jump to arbitrary page	Yes	No — sequential only
Stable under concurrent writes	No — rows can shift between pages	Yes
Needs	Nothing special	An index on the sort/tie-break columns
Typical use	Admin tables, small result sets, numbered pagination UI	Infinite scroll, APIs, large tables

INTERVIEW QUESTIONS — PART TEN

1. **Is result order guaranteed without ORDER BY?** Beginner No — never rely on it.
2. **In LIMIT 10, 20, which number is the offset?** Intermediate The first — 10 rows are skipped, then 20 returned. This is reversed from the OFFSET keyword form.
3. **Why does deep pagination get slower?** Intermediate OFFSET must read and discard every preceding row before it can start returning results.
4. **How would you build "load more" for a feed of millions of rows?** Advanced Keyset pagination on an indexed, unique (or tie-broken) sort key — pass the last seen key as a cursor.
5. **Find the second-highest salary without a window function.** Advanced

```
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;
— DISTINCT avoids skipping a tie.
```

PRACTICE — PART TEN

1. List the five most recent orders, then the next five, using OFFSET and then using a keyset cursor. Beginner
2. Find the third-highest paid employee per your dataset, handling ties correctly. Intermediate
3. Sort tickets by a custom priority order without ENUM, using FIELD(). Intermediate

PART ELEVEN

Operators and precedence

Every operator SQL offers, and the order it applies them in when you don't say otherwise.

11.1 Arithmetic operators

```
SELECT 7 + 3, 7 - 3, 7 * 3, 7 / 3, 7 DIV 3, 7 % 3, MOD(7,3);
--    10    4    21   2.3333   2      1    1
SELECT 10 / 0;         -- NULL, not an error (with default sql_mode)
SELECT 10 DIV 0;       -- NULL
SELECT -5 % 3, 5 % -3; -- -2, 2  : sign follows the dividend
```

/ always produces a decimal result, even for two integers; DIV is integer division, truncated toward zero. Division by zero is NULL in MySQL by default (it becomes an error only under the ERROR_FOR_DIVISION_BY_ZERO mode) — a genuine portability trap, since PostgreSQL and SQL Server raise an error immediately.

11.2 Comparison operators

OPERATOR	MEANING	NULL-SAFE?
= <> / !=	Equal / not equal	No — NULL in, NULL/UNKNOWN out
< <= > >=	Ordering	No
<=>	Null-safe equal	Yes — MySQL extension
BETWEEN … AND	Inclusive range	No
IN (…)	Membership	No
LIKE	Pattern match	No
IS [NOT] NULL	Null test	Yes — the only correct way to test NULL

11.3 Logical operators

Covered fully in Part 9: AND, OR, NOT, and the three-valued truth tables. Also standard: XOR in MySQL (true when exactly one side is true) and ALL / ANY / SOME for comparing against a subquery's rows (Part 16).

11.4 String and set operators

```
SELECT 'Sql' || ' ' || 'Rocks';        -- '' with PIPES_AS_CONCAT mode only
SELECT CONCAT('Sql', ' ', 'Rocks');    -- the portable MySQL way
SELECT 5 & 3, 5 | 2, 5 ^ 1, ~5, 5 << 1, 5 >> 1;  -- bitwise: AND OR XOR NOT
SHL SHR
```

MySQL uses || for logical OR by default, not concatenation — the opposite of Oracle, PostgreSQL and SQL Server, where || concatenates strings. Always use CONCAT() in MySQL to avoid the ambiguity entirely.

11.5 Operator precedence

```
Highest
  1.  ( )                          parentheses
  2.  *  /  DIV  %  MOD             multiplicative
  3.  +  -                          additive (and unary +/-)
  4.  <<  >>                       bit shifts
  5.  &                            bitwise AND
  6.  |  ^                          bitwise OR / XOR
  7.  =  <=>  <>  <  <=  >  >=  IN  LIKE  BETWEEN  IS   comparison
  8.  NOT
  9.  AND  (&&)
 10.  XOR
 11.  OR  (||)
 12.  =  (assignment, in SET only)
Lowest
```

Two consequences worth memorising. salary > 50000 AND dept_id = 10 OR dept_id = 20 means *(high earners in dept 10) or (anyone in dept 20)* — almost never what a beginner intends when they meant "high earners in 10 or 20". And NOT salary > 50000 AND dept_id = 10 negates only the salary test, not the whole expression, because NOT binds to the single condition after it.

> IMPORTANT
>
> Precedence is not something to master from memory under pressure — it is something to make irrelevant. Parenthesise every WHERE clause that mixes AND with OR, or that mixes NOT with anything besides a single simple condition. The one-line rule: *if you had to think about precedence to read it, so will the next person, so add parentheses.*

INTERVIEW QUESTIONS — PART ELEVEN

1. **What does `a = 1 AND b = 2 OR c = 3` mean?** Intermediate `(a=1 AND b=2) OR c=3` — AND binds tighter.
2. **Does `||` concatenate in MySQL?** Intermediate Only under PIPES_AS_CONCAT; by default it is logical OR. Use CONCAT().
3. **What does `7 / 2` return? What does `7 DIV 2` return?** Beginner `3.5000` and `3`.
4. **Is division by zero an error in MySQL?** Advanced By default, no — it returns NULL, unlike PostgreSQL and SQL Server which raise an error.

PRACTICE — PART ELEVEN

1. Predict the output of five mixed AND/OR expressions on paper, then run them. Beginner
2. Rewrite three ambiguous WHERE clauses from earlier parts with full parentheses. Intermediate